



POST AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY



Ôn tập và bài tập: Các thành phần phần cứng

Faculty: Computer science – IT1

Instructor: Viet Hoang Pham

Email: vietph@ptit.edu.vn

Phone: 0945296388

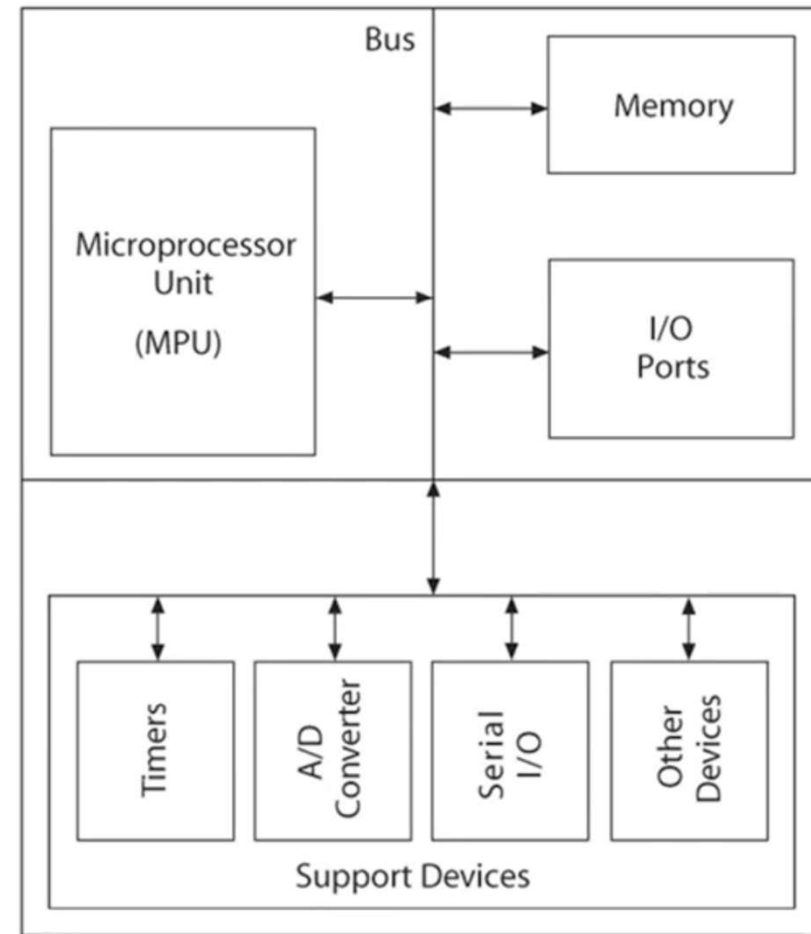
- Bộ xử lý trung tâm -CPU
- Bộ nhớ và thiết kế bộ nhớ
- Ghép nối với thiết bị ngoại vi
- Vi xử lý 8051: Kiến trúc, bộ nhớ và giao diện thực tế

❖ Vi xử lý (Microprocessor)

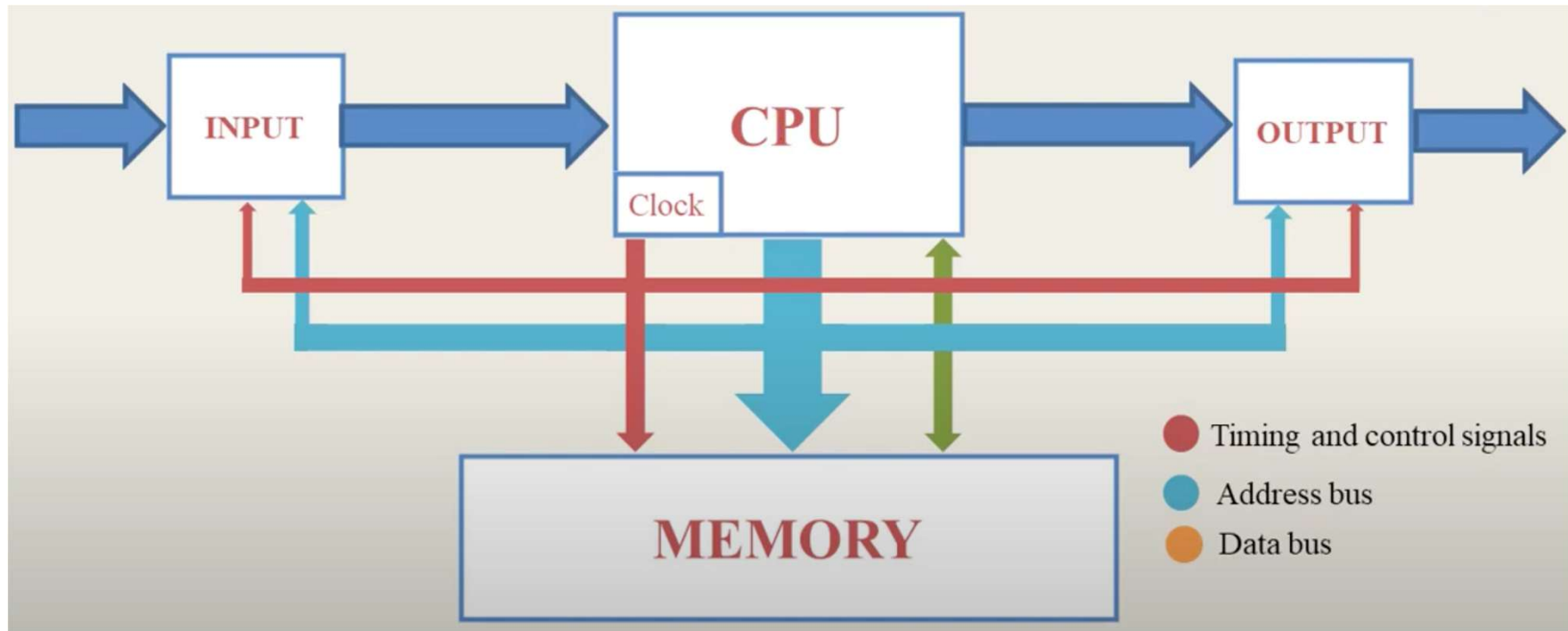
- Đặc điểm
 - Chỉ chứa CPU
 - Cần RAM, ROM, I/O bên ngoài
 - Hiệu năng cao, chạy hệ điều hành (OS)
- Dùng trong:
 - PC
 - Server
 - Hệ thống tính toán nặng
- Ví dụ: Intel x86, AMD Ryzen

❖ Vi xử lý (Microprocessor)

- Tích hợp sẵn:
 - CPU, RAM, Flash,
 - Timer, ADC, GPIO,
 - UART/SPI/I2C
- Dùng trong:
 - IoT,
 - Robot,
 - Thiết bị gia dụng,
 - Hệ điều khiển công nghiệp
- Ví dụ: STMicroelectronics STM32, Microchip Technology PIC, Espressif Systems ESP32



Tổng quan phần cứng HTN

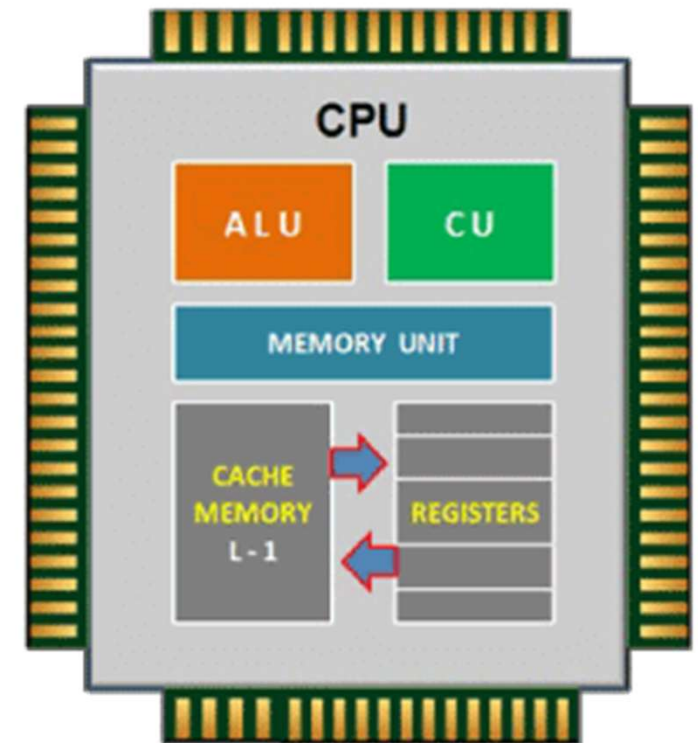


- ❖ **CPU** (Bộ xử lý trung tâm): Thực hiện lệnh và xử lý dữ liệu
- ❖ **Memory**: Bộ nhớ chương trình và dữ liệu
- ❖ **Ngoại vi**: Input / output devices
- ❖ **Bus**: truyền tín hiệu giữa các thành phần

❖ Sơ đồ khối cơ bản CPU gồm:

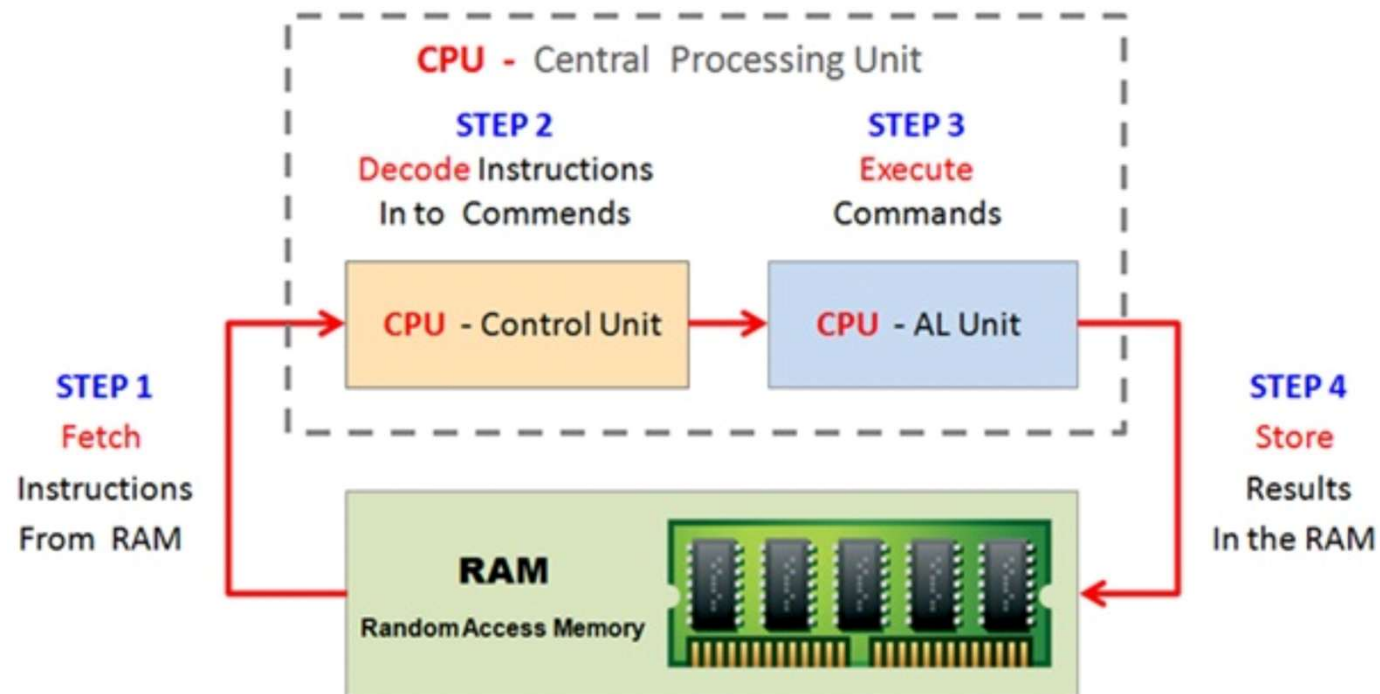
- **ALU**: Đơn vị tính toán và logic
- **CU**: Đơn vị điều khiển
- **Memory Unit**: Một số bộ nhớ được tích hợp trong CPU để hỗ trợ lưu trữ và tính toán nhanh. Gồm các thanh ghi và Cache.

Central Processing Unit



❖ Quá trình thực thi một lệnh:

1. Nạp lệnh từ bộ nhớ vào CU
2. CU Giải mã lệnh
3. ALU thực thi lệnh
4. Kết quả được lưu vào bộ nhớ



❖ Kiến trúc tập lệnh ISA (Instruction Set Architecture):

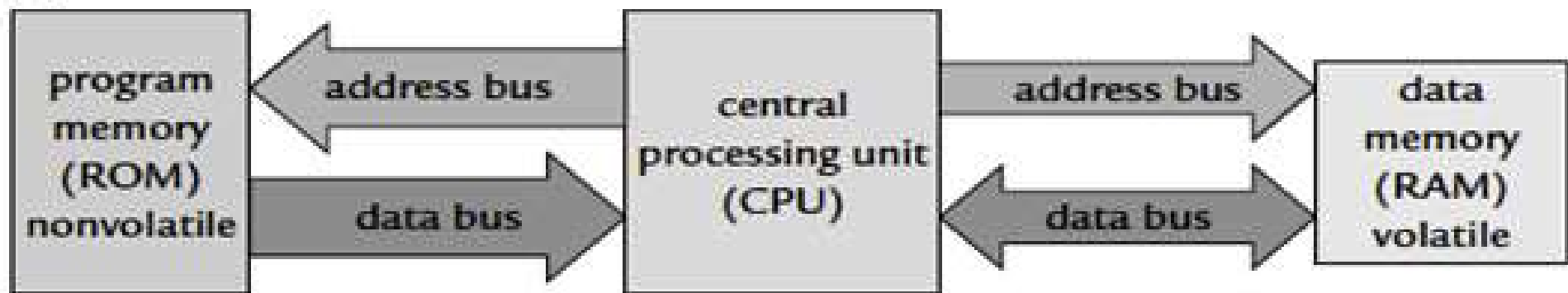
- Kiến trúc tập lệnh quy định cách thức CPU tương tác với dữ liệu và bộ nhớ thông qua:
 - Chế độ địa chỉ hóa (Addressing Modes)
 - Tổ chức lưu trữ dữ liệu
 - Tập thanh ghi (Registers)
- Trong thiết kế hệ thống nhúng, việc lựa chọn kiến trúc tập lệnh ảnh hưởng trực tiếp đến:
 - Hiệu năng xử lý
 - Tối ưu hóa mã nguồn
 - Công cụ phát triển
- Có hai loại kiến trúc tập lệnh:
 - CISC (Complex Instruction Set Computing)
 - RISC (Reduce Instruction Set Computing)

❖ Kiến trúc tập lệnh ISA (Instruction Set Architecture):

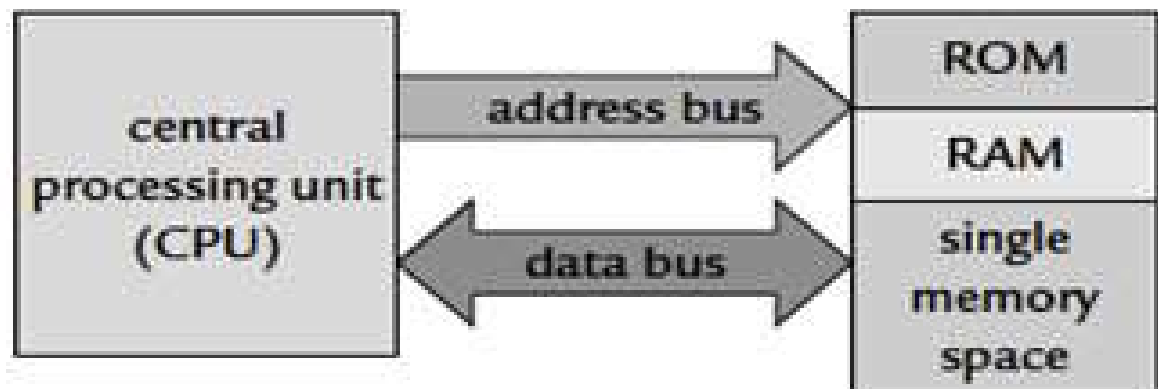
Đặc điểm	CISC (Tập lệnh phức hợp)	RISC (Tập lệnh rút gọn)
Truy cập bộ nhớ	Bất kỳ lệnh nào cũng có thể truy cập	Chỉ lệnh Load và Store
Độ dài lệnh	Thay đổi tùy theo độ phức tạp	Cố định
Thời gian thực hiện	Nhiều nhịp chu kỳ	Thường chỉ một nhịp chu kỳ
Thanh ghi	Chỉ có một tập thanh ghi	Có nhiều tập thanh ghi
Cơ chế thực thi	Thông dịch qua vi mã (microcode)	Phần cứng thực thi trực tiếp
Hỗ trợ Pipeline	Không hỗ trợ (vẫn có thể)	Hỗ trợ mạnh mẽ
Kích thước mã	Nhỏ gọn, tối ưu bộ nhớ	Lớn hơn

❖ Kiến trúc hệ thống nhúng

(a) Harvard architecture



(b) von Neumann architecture

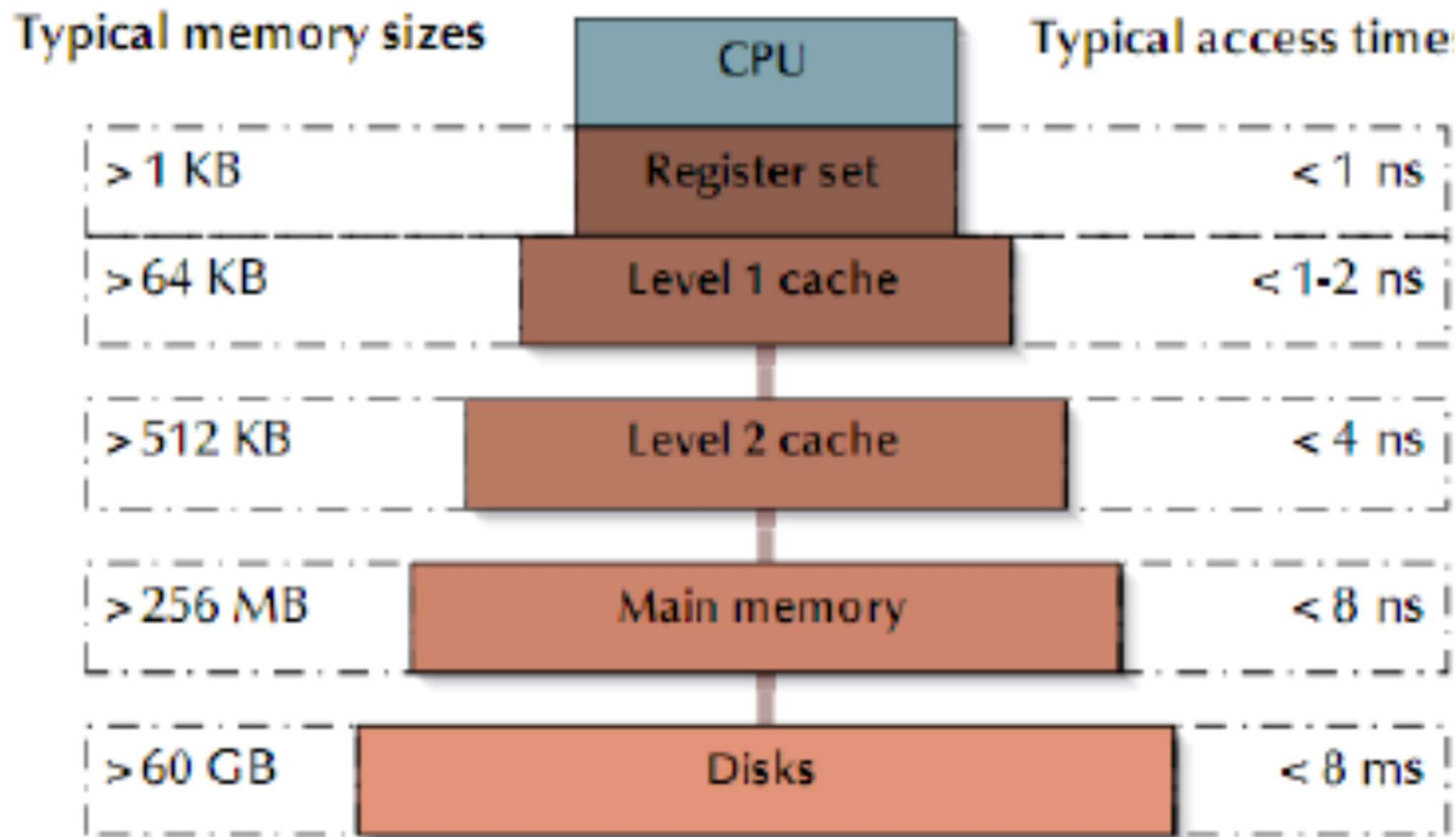


❖ Bộ nhớ là phương tiện lưu trữ thông tin bao gồm chương trình và số liệu.

Một số thông tin cơ bản về bộ nhớ máy tính:

- Dung lượng được xác định theo công thức: $\text{Dung lượng} = 2^n \times m$. Trong đó: n là số bit địa chỉ, tức là số đường địa chỉ mà CPU dùng để chọn ô nhớ, m là độ dài của mỗi ô nhớ.
- Thời gian thâm nhập bộ nhớ. Thời gian thâm nhập, ký hiệu là t_a , là khoảng thời gian từ lúc CPU đặt địa chỉ lên BUS cho đến khi dữ liệu thực sự xuất hiện trên BUS dữ liệu.
- Chu kỳ đọc (t_{RC}) và chu kỳ ghi (t_{WC}) cho biết: khoảng thời gian ngắn nhất giữa hai lần truy cập liên tiếp vào cùng một ô nhớ.
- trạng thái đợi – Wait State, ký hiệu là t_w : là những chu kỳ xung nhịp mà CPU buộc phải đứng lại, chờ bộ nhớ trả dữ liệu về.

❖ Mô hình phân cấp bộ nhớ:



❖ **Memory map** (bản đồ bộ nhớ) là cách **phân bố không gian địa chỉ** để CPU có thể truy cập: bộ nhớ chương trình, bộ nhớ dữ liệu, thanh ghi ngoại vi, thiết bị I/O

- CPU chỉ làm việc với: địa chỉ (address) và dữ liệu (data)
- Nếu không có memory map, CPU không thể giao tiếp phần cứng.
- CPU có N bit address bus $\rightarrow$ Không gian địa chỉ $= 2^N$
- Ví dụ đơn giản cho CPU 16-bit
 - 0000–7FFF Flash
 - 8000–9FFF RAM
 - A000–A0FF Peripheral
 - A100–FFFF Free
- Memory map cho phép truy cập ngoại vi như biến, không cần lệnh I/O riêng, và cho phép thêm thiết bị bằng ánh xạ địa chỉ.

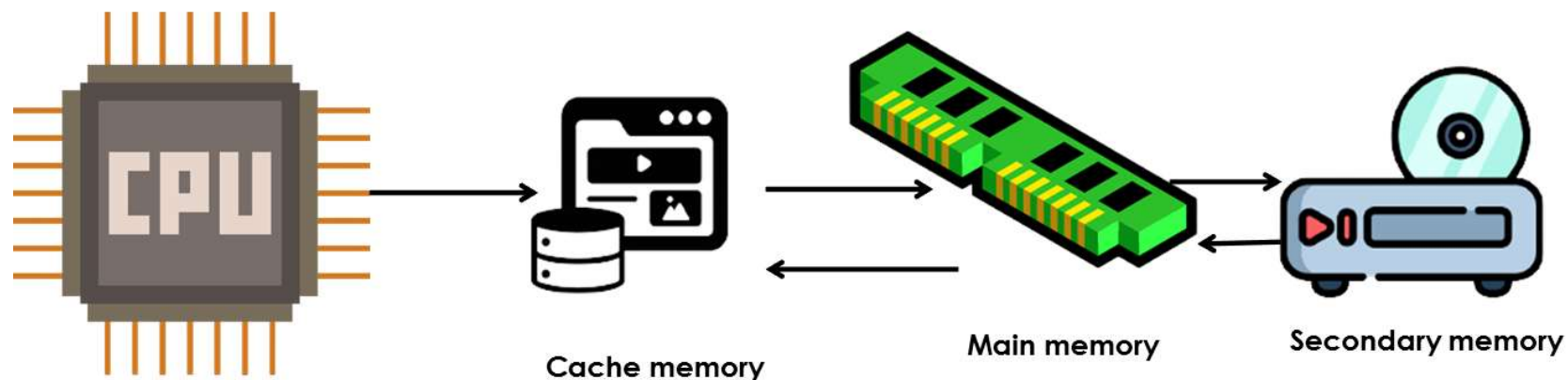
❖ **Memory map** (bản đồ bộ nhớ) là cách **phân bố không gian địa chỉ** để CPU có thể truy cập: bộ nhớ chương trình, bộ nhớ dữ liệu, thanh ghi ngoại vi, thiết bị I/O

- CPU chỉ làm việc với: địa chỉ (address) và dữ liệu (data)
- Nếu không có memory map, CPU không thể giao tiếp phần cứng.
- CPU có N bit address bus $\rightarrow$ Không gian địa chỉ $= 2^N$
- Ví dụ đơn giản cho CPU 16-bit
 - 0000–7FFF Flash
 - 8000–9FFF RAM
 - A000–A0FF Peripheral
 - A100–FFFF Free
- Memory map cho phép truy cập ngoại vi như biến, không cần lệnh I/O riêng, và cho phép thêm thiết bị bằng ánh xạ địa chỉ.

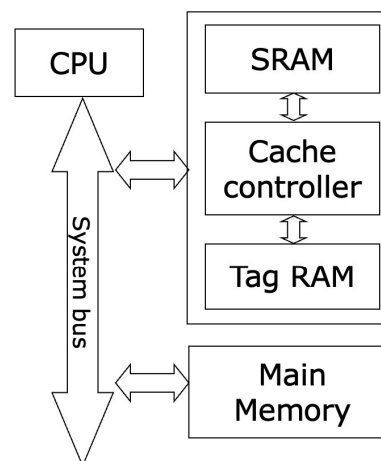
❖ Thanh ghi (Register):

- Thanh ghi là bộ nhớ siêu nhanh nằm trong CPU.
- Dùng để: Lưu toán hạng, Lưu địa chỉ, Điều khiển hoạt động CPU
- Các loại thanh ghi quan trọng
 - Thanh ghi đặc biệt: thanh ghi tích lũy,...
 - Thanh ghi đa dụng: Lưu dữ liệu tạm
 - Program Counter (PC): Lưu địa chỉ lệnh tiếp theo
 - Instruction Register (IR): Chứa OpCode
 - Stack Pointer (SP): Quản lý stack
 - Status Register: Cờ Zero, Carry, Overflow
 - Control Registers: Điều khiển ngoại vi

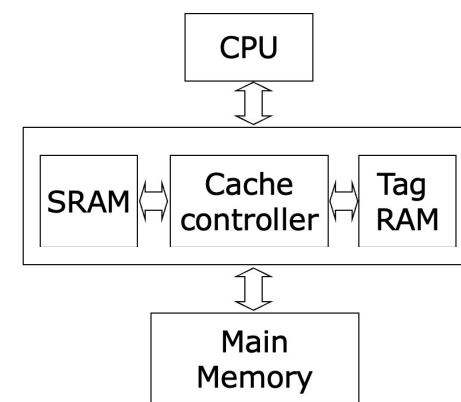
- ❖ Trong sơ đồ phân cấp bộ nhớ của máy tính, cache là một thành phần nhớ nằm giữa CPU và bộ nhớ chính (RAM). Nó đóng vai trò trung gian, giúp thu hẹp khoảng cách tốc độ rất lớn giữa CPU và RAM.
- ❖ Cache hoạt động như một bộ trung chuyển dữ liệu:
 - Khi CPU cần dữ liệu hoặc lệnh, hệ thống sẽ ưu tiên tìm trong cache trước.
 - Nếu dữ liệu đã có trong cache, CPU có thể truy cập rất nhanh, không cần chờ RAM.
 - Nếu dữ liệu chưa có trong cache, dữ liệu sẽ được lấy từ bộ nhớ chính, đưa vào cache, rồi mới chuyển tới CPU.
 - Quá trình này diễn ra tự động.



- ❖ **Cache Hit:** CPU yêu cầu một lệnh hoặc dữ liệu, và cache đã có sẵn dữ liệu đó.
 - Xác suất xảy ra hit được gọi là hệ số hit, ký hiệu là H , với: $0 \leq H \leq 1$
- ❖ **Cache Miss:** CPU yêu cầu dữ liệu hoặc lệnh, nhưng cache không có dữ liệu đó.
 - Khả năng xảy ra miss được gọi là hệ số miss, bằng: $1-H$
- ❖ có hai kiến trúc bố trí cache chính
 - Kiến trúc look-aside (kiểu ngang hàng – bình đẳng): CPU, cache và bộ nhớ chính cùng được nối song song trên bus hệ thống.
 Khi CPU phát ra yêu cầu truy cập bộ nhớ, cache và bộ nhớ chính đồng thời kiểm tra địa chỉ được yêu cầu.
 - Kiến trúc look-through (kiểu phân chia – phân cấp): CPU chỉ làm việc trực tiếp với cache, cache đóng vai trò trung gian bắt buộc giữa CPU và bộ nhớ chính.



Look aside

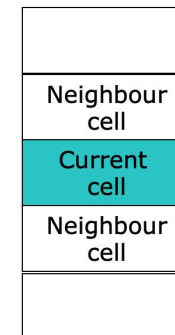


Look through

- ❖ Bộ nhớ cache được xem là một bộ nhớ thông minh bởi vì nó không chỉ lưu trữ thụ động dữ liệu, mà còn chủ động hỗ trợ CPU trong quá trình xử lý.
- ❖ Cache có khả năng đoán trước nhu cầu về lệnh và dữ liệu của CPU.

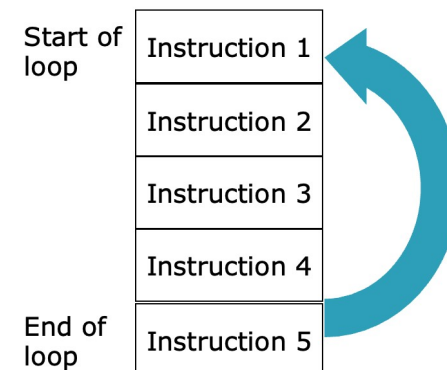
- Nguyên lý cục bộ về không gian (Spatial locality):

Khi CPU truy cập đến một ô nhớ nào đó, thì khả năng cao là CPU sẽ sớm truy cập đến các ô nhớ lân cận.



- Nguyên lý cục bộ về thời gian (Temporal locality):

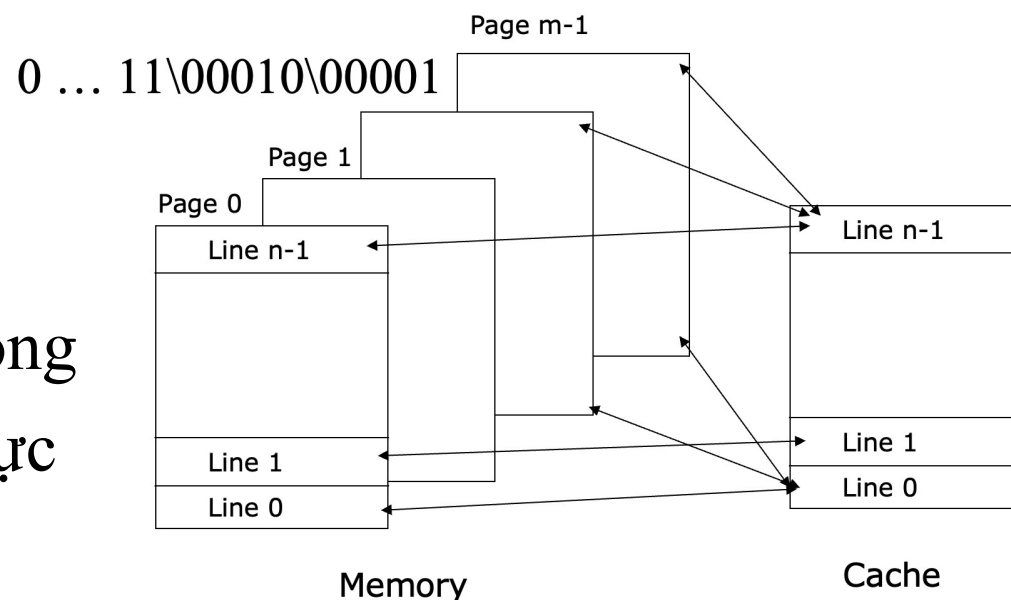
Nếu CPU vừa truy cập một dữ liệu hay lệnh nào đó, thì khả năng cao là dữ liệu hay lệnh đó sẽ được truy cập lại trong tương lai gần.



- ❖ Ba cách tổ chức cache: ánh xạ trực tiếp, ánh xạ kết hợp đầy đủ, và ánh xạ tập kết hợp

❖ Ánh xạ trực tiếp:

- Cache được chia thành n khối, còn gọi là n dòng cache (cache lines) từ Line 0 tới Line $n-1$
- Bộ nhớ:
 - Bộ nhớ chính được chia thành m trang, từ page 0 tới page $m-1$
 - Mỗi trang bộ nhớ có kích thước đúng bằng kích thước của cache
 - mỗi trang cũng được chia thành n dòng
- Ánh xạ:
 - Line i của bất kỳ trang nào trong bộ nhớ chính sẽ được ánh xạ trực tiếp tới Line i của cache.



❖ Trường địa chỉ trong ánh xạ trực tiếp:

- Tag (bit): xác định trang (page) của bộ nhớ chính
- Line (bit): xác định dòng (line) trong cache
- Word (bit): xác định từ nhớ (word) bên trong một dòng cache

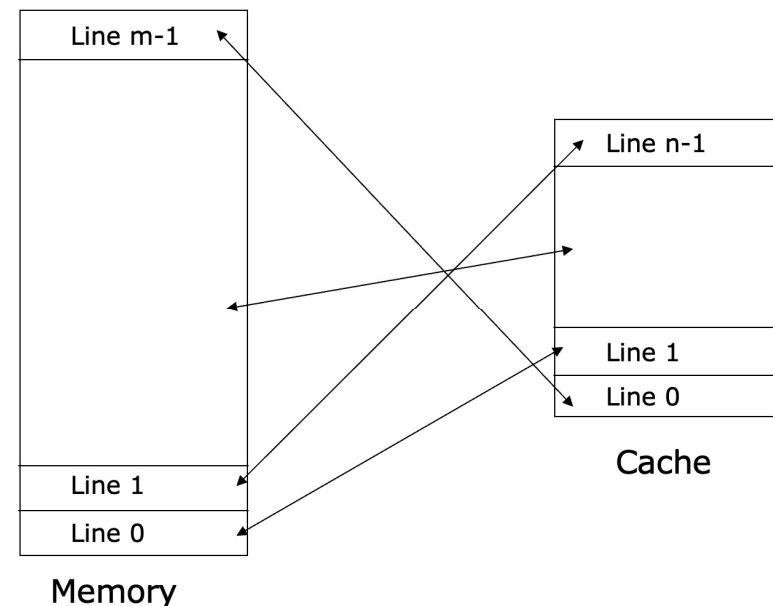
Tag	Line	Word
-----	------	------

How to determine:

- $Tag + Line + Word = Address_Bus_Size(bit) = \log_2 \left(\frac{M_{Memory}}{[Cell]} \right)$
- Number of memory pages: $P = \frac{M_{Memory}}{[M_{Cache}]} \Rightarrow tag = \log_2 (P)$
- Number of lines in a page: $L = \frac{M_{Cache}}{[Line]} \Rightarrow line = \log_2 (L)$
- Number of memory cells in a line: $W = \frac{Line}{[Cell]} \Rightarrow word = \log_2 (W)$

❖ Ánh xạ kết hợp đầy đủ

- Cache: được chia thành n dòng (line), còn gọi là block, từ Line 0 tới Line $n-1$
- Bộ nhớ:
 - được chia thành m dòng, từ Line 0 tới Line $m-1$
 - Kích thước của một dòng bộ nhớ chính bằng đúng kích thước một dòng cache
 - $m \gg n$
- Ánh xạ: Line i của bộ nhớ có thể ánh xạ tới Line j bất kì trong cache



❖ Trường địa chỉ trong ánh xạ kết hợp đầy đủ

- Tag (bit): địa chỉ của dòng (line) trong bộ nhớ chính
- Word (bit): địa chỉ của từ nhớ (word) bên trong một dòng (line)

Tag	Word
-----	------

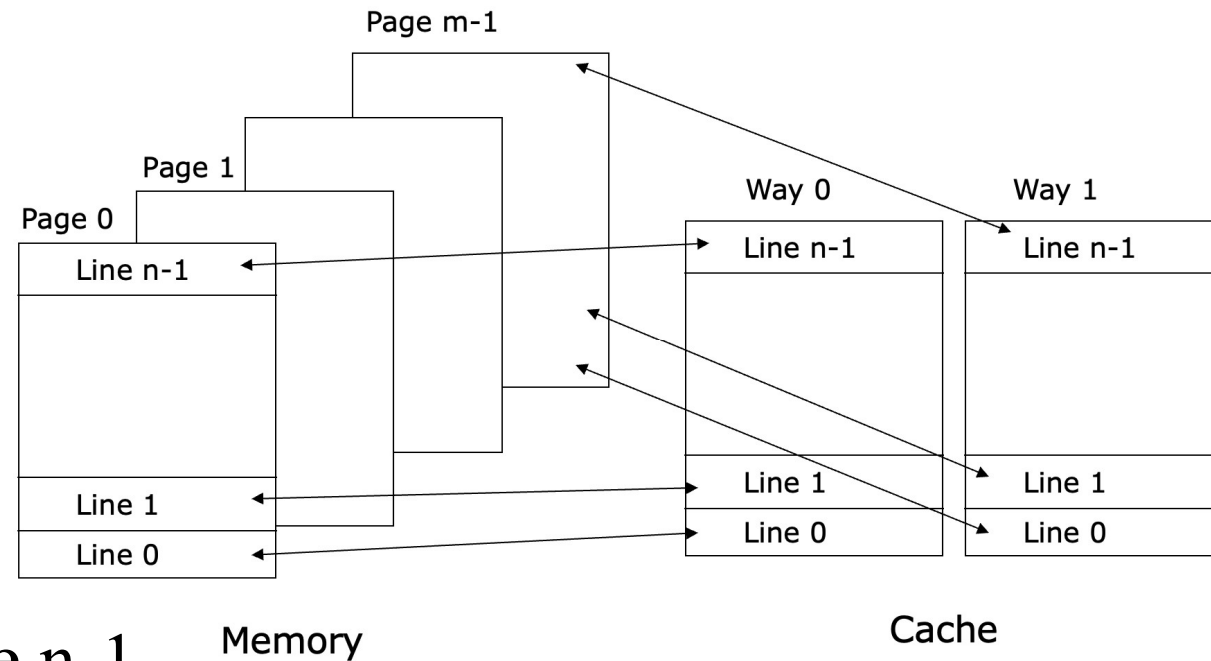
Where

- $\text{Tag} + \text{Word} = \text{Address_Bus_Size (bit)} = \log_2 \left(\frac{M_{\text{Memory}}}{[Cell]} \right)$
- Number of memory lines: $L = \frac{M_{\text{Memory}}}{[Line]} \Rightarrow \text{tag} = \log_2 (L)$
- Number of memory cells in a line:

$$W = \frac{[Line]}{[Cell]} \Rightarrow \text{Word} = \log_2 (W)$$

❖ Ánh xạ tập kết hợp

- Cache được chia thành k đường (ways) có kích thước bằng nhau
- Mỗi way lại được chia thành n dòng (line hoặc block), từ Line 0 tới Line n-1



- Memory:
 - Bộ nhớ được chia thành m trang (page), từ page 0 tới page m-1
 - Kích thước của mỗi page bằng kích thước của một way cache
 - Mỗi trang bộ nhớ cũng được chia thành n dòng, từ Line 0 tới Line n-1

❖ Ánh xạ tập kết hợp

- Các Pages được ánh xạ tới các Ways (full mapping):
 - Mỗi page của bộ nhớ chính: Có thể được ánh xạ vào bất kỳ way nào của cache. Không có sự cố định page phải vào way nào.
 - Line i của page: Sẽ được ánh xạ cố định tới Line i của way j

Tag	Set	Word
-----	-----	------

❖ Trường địa chỉ trong ánh xạ tập kết hợp

- Tag (bit): địa chỉ của page trong bộ nhớ chính
- Set (bit): địa chỉ của line (dòng) trong một page
- Word (bit): địa chỉ của word (ô nhớ) trong một line

Tag	Set	Word
-----	-----	------

where

- $\text{Tag} + \text{Set} + \text{Word} = \text{Address_Bus_Size (bit)} = \log_2 \left(\frac{M_{\text{Memory}}}{[\text{Cell}]} \right)$
- Number of memory pages: $P = \frac{M_{\text{Memory}} \times \text{Way}}{M_{\text{Cache}}} \Rightarrow \text{tag} = \log_2 (P)$
- Number of lines in a page: $L = \frac{M_{\text{Cache}}}{[\text{Word}] \times \text{Way}} \Rightarrow \text{Set} = \log_2 (L)$
- Number of memory cells in a line: $W = \frac{[\text{Line}]}{[\text{Cell}]} \Rightarrow \text{Word} = \log_2 (W)$

❖ Exercise: Find the size of the address field using direct mapping, given:

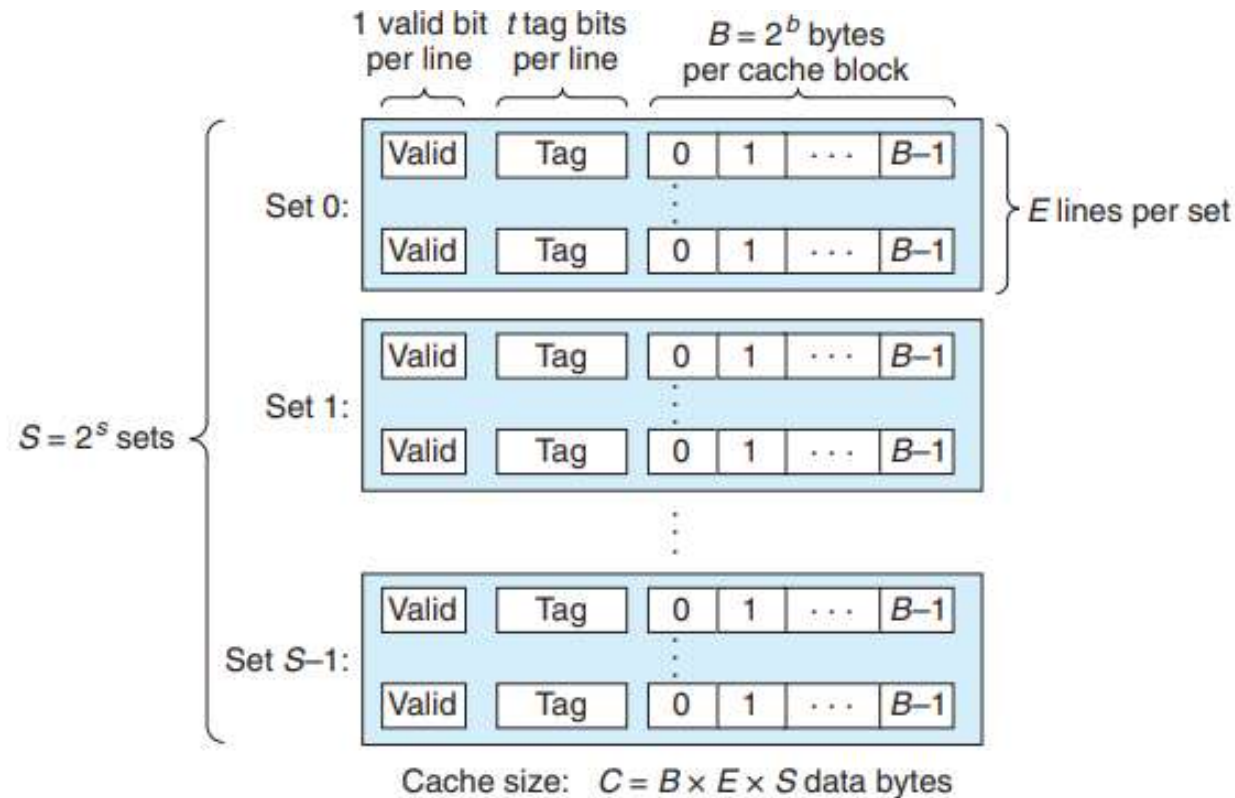
- Memory size: 4GB
- Cache size: 1KB
- Line size: 32 bytes

- ❖ Exercise: Find the size of the address field using Fully associative mapping, knowing:
- Memory size: 4GB
 - Cache size: 1KB
 - Size of a line: 32 bytes

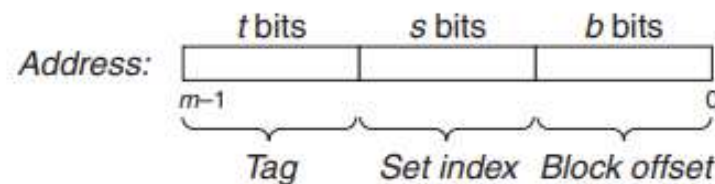
- ❖ Exercise: Find the size of the address field using associative set mapping:
- Memory size: 4GB
 - Cache size: 1KB, 2 ways
 - Size of a line: 32 bytes

- ❖ Tính toán số bit trong bộ đệm:
1. Tính tổng số bit cần thiết cho một bộ nhớ cache tổ chức theo ánh xạ trực tiếp có kích thước là 64 KByte và một dòng cache có kích thước là 1 từ nhớ, giả sử bus địa chỉ 32 bit?
 2. Tổng số bit sẽ cần là bao nhiêu cho bộ nhớ cache sử dụng tập kết hợp được thiết lập 4 way có kích thước là 64 KByte và một dòng cache có kích thước là 1 từ nhớ, giả sử bus địa chỉ 32 bit?
 3. Tính tổng số bit cần thiết cho một bộ nhớ cache tổ chức theo ánh xạ trực tiếp kích thước là 64 KB và một dòng cache có kích thước là 8 từ nhớ, giả sử bus địa chỉ 32 bit?

❖ Tính toán số bit trong bộ đệm:



(a)

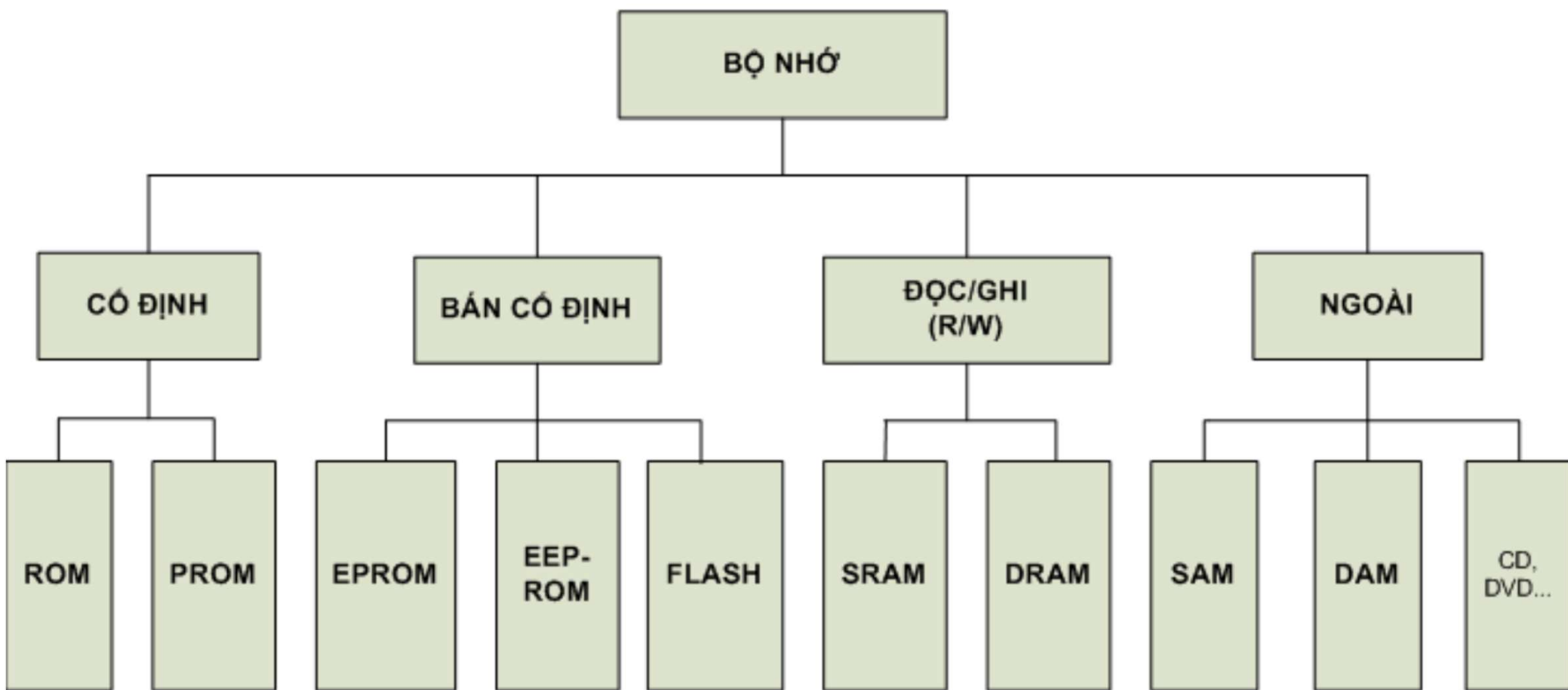


❧ Calculate the total number of bits required for a direct-mapped cache of size 64 KB and a cache line of size 8 memory words, assuming a 32-bit address bus?

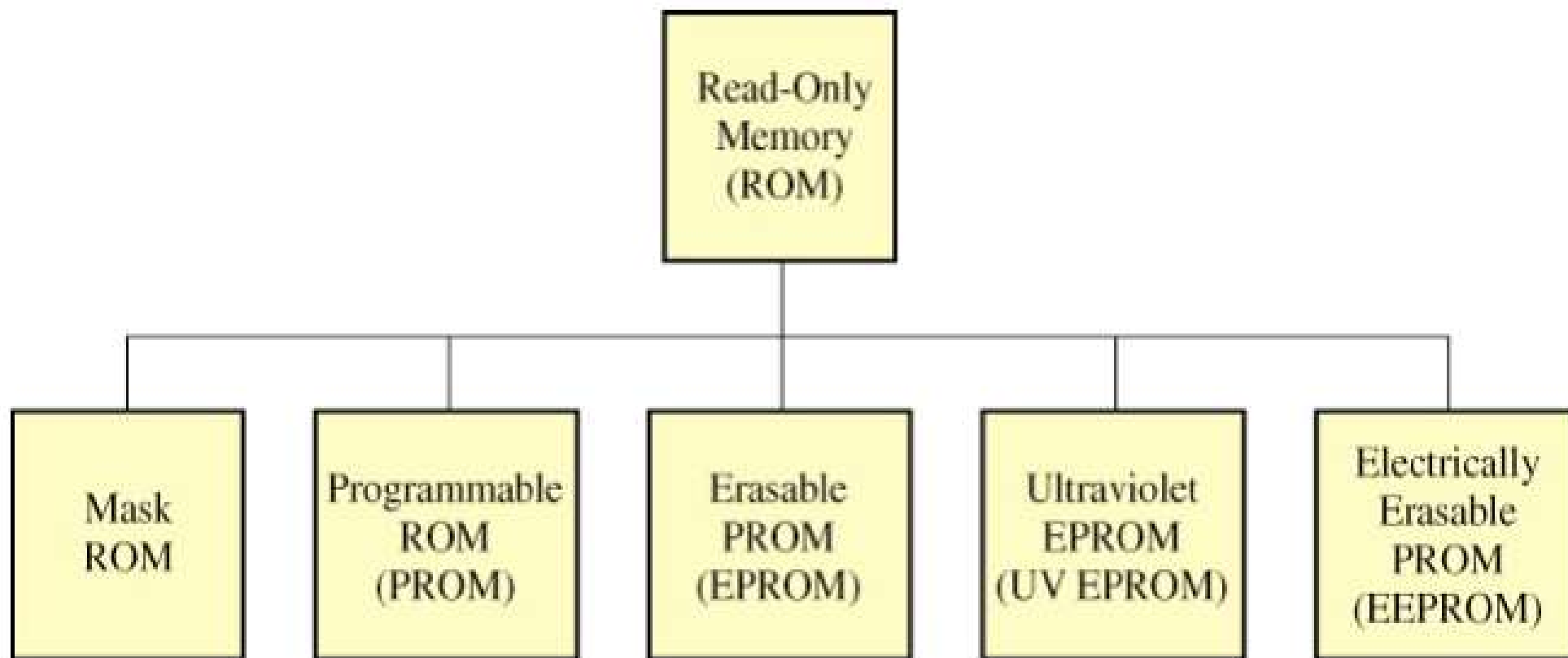
Solution:

- We have 1 cache line $8 \times 32 \text{ bits} = 32 \text{ bytes} \Rightarrow \text{word} = 5 \text{ bits}$
- The number of cache lines is $L = [\text{Cache}]/[\text{Word}] = 64 \text{ Kbytes}/32 \text{ bytes} = 2^{11} \Rightarrow \text{line} = 11 \text{ bits}$
- $\text{tag} = \text{address_size} - \text{line} - \text{word} = 32 - 11 - 5 = 16 \text{ bits}$
- The number of bits required for one cache line is:
 $\text{bits/block} = \text{data bits} + \text{tag bits} + \text{valid bit} = 8 \times 32 + 16 + 1 = 273 \text{ bits}$
- The total number of bits required for the cache memory is $= L \times \text{bits/block} = 2^{12} \times 273 \text{ bits} = 68.25 \text{ Kbytes}$

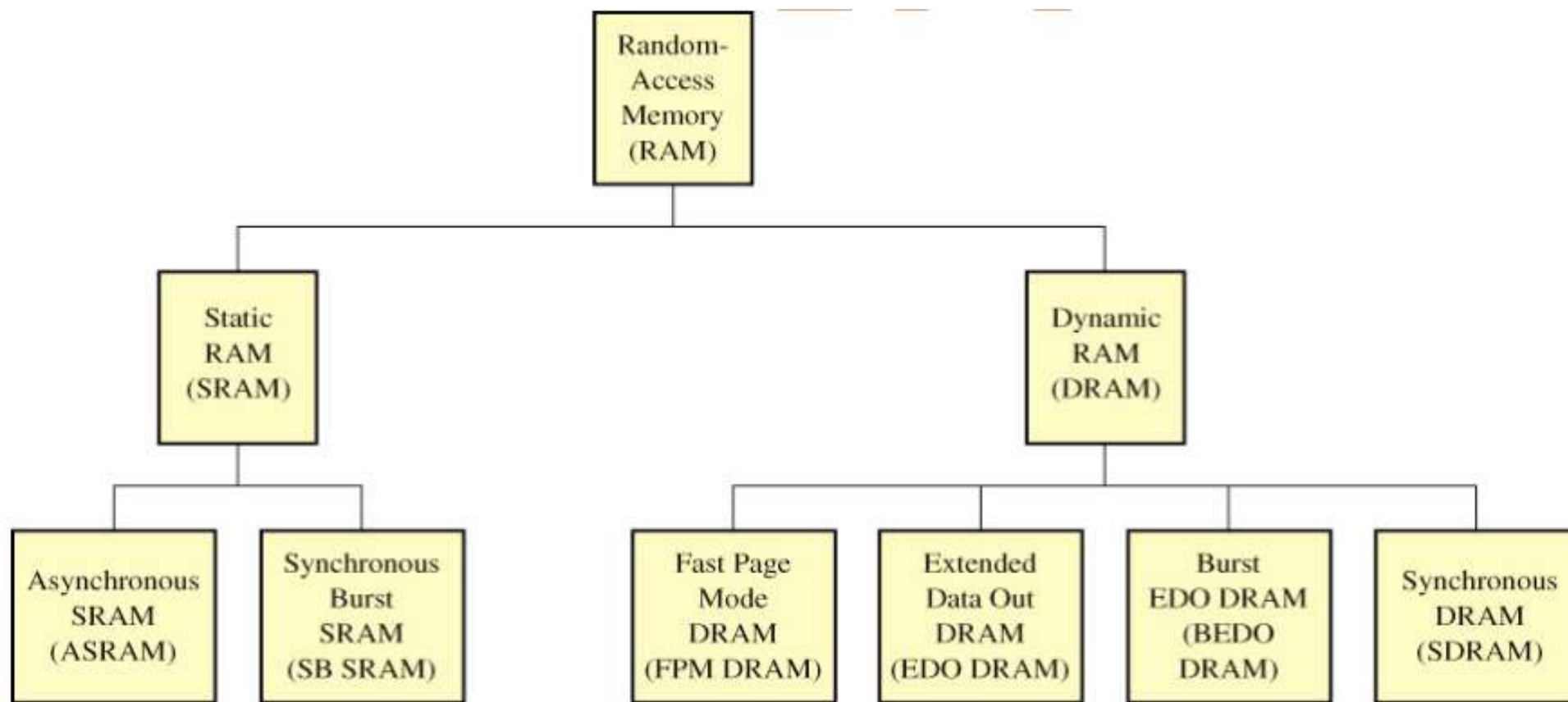
❖ Các loại bộ nhớ:



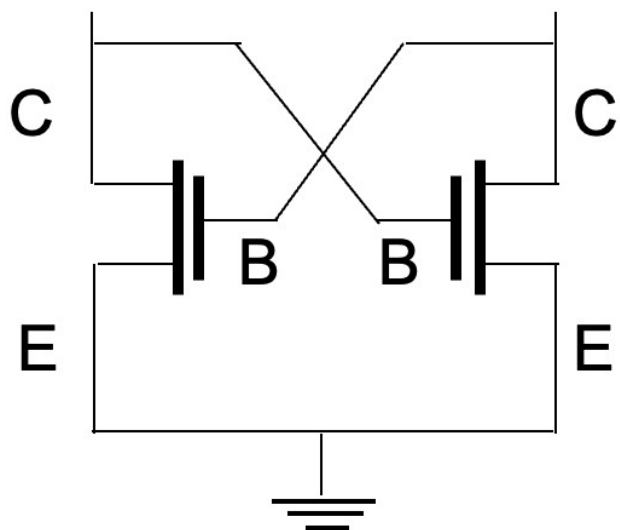
❖ Các loại bộ nhớ ROM:



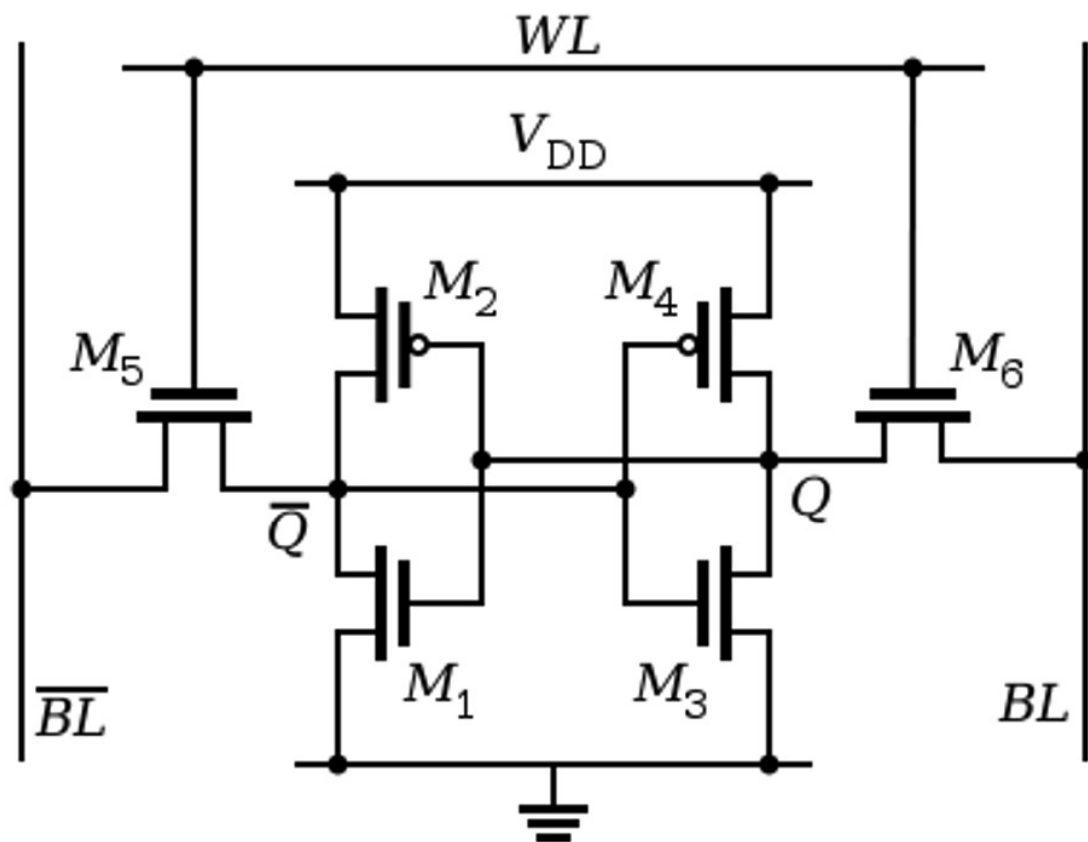
❖ Các loại bộ nhớ RAM:



❖ SRAM: giá thành đắt, thường dùng làm cache

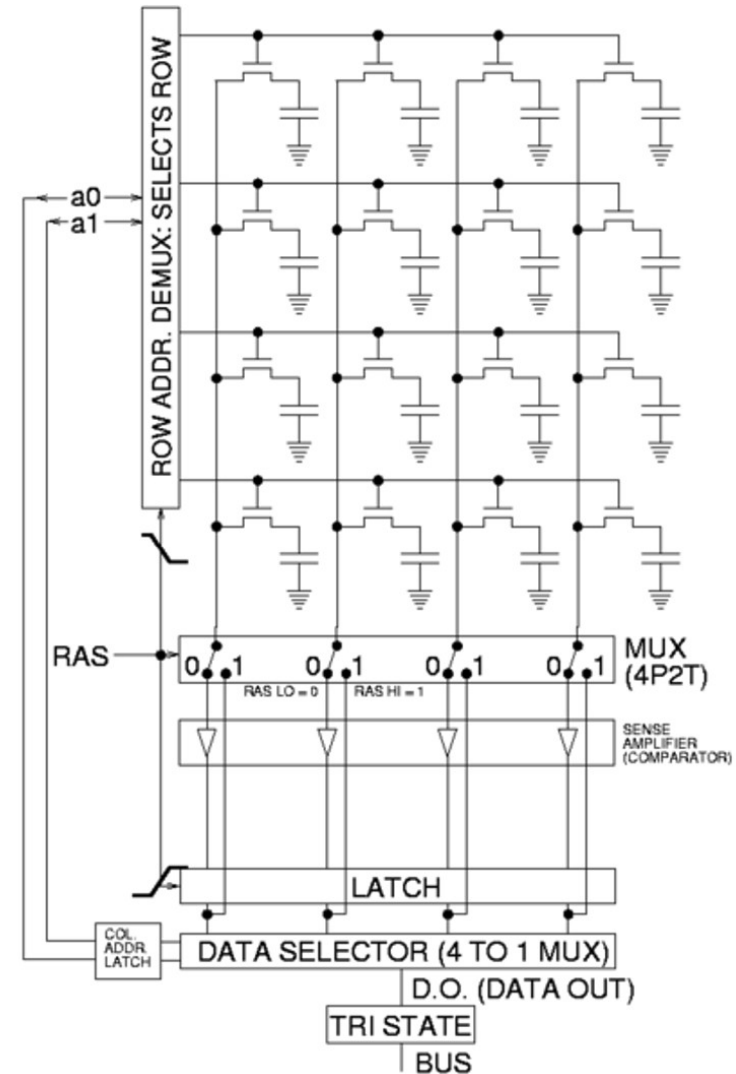
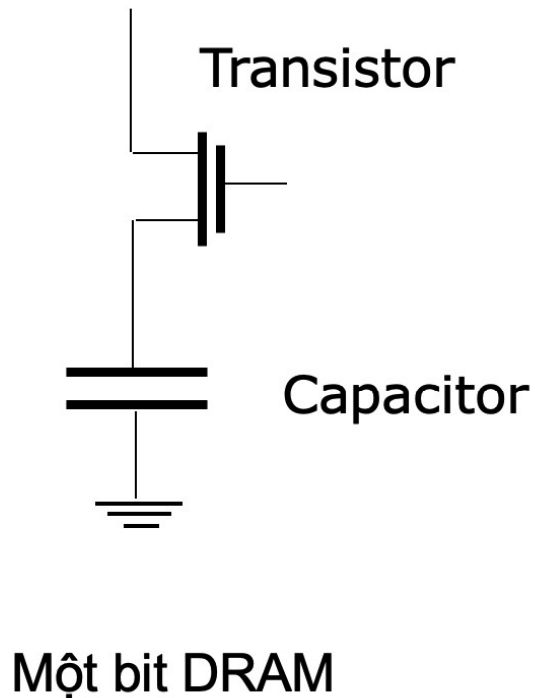


Một mạch lật đơn giản



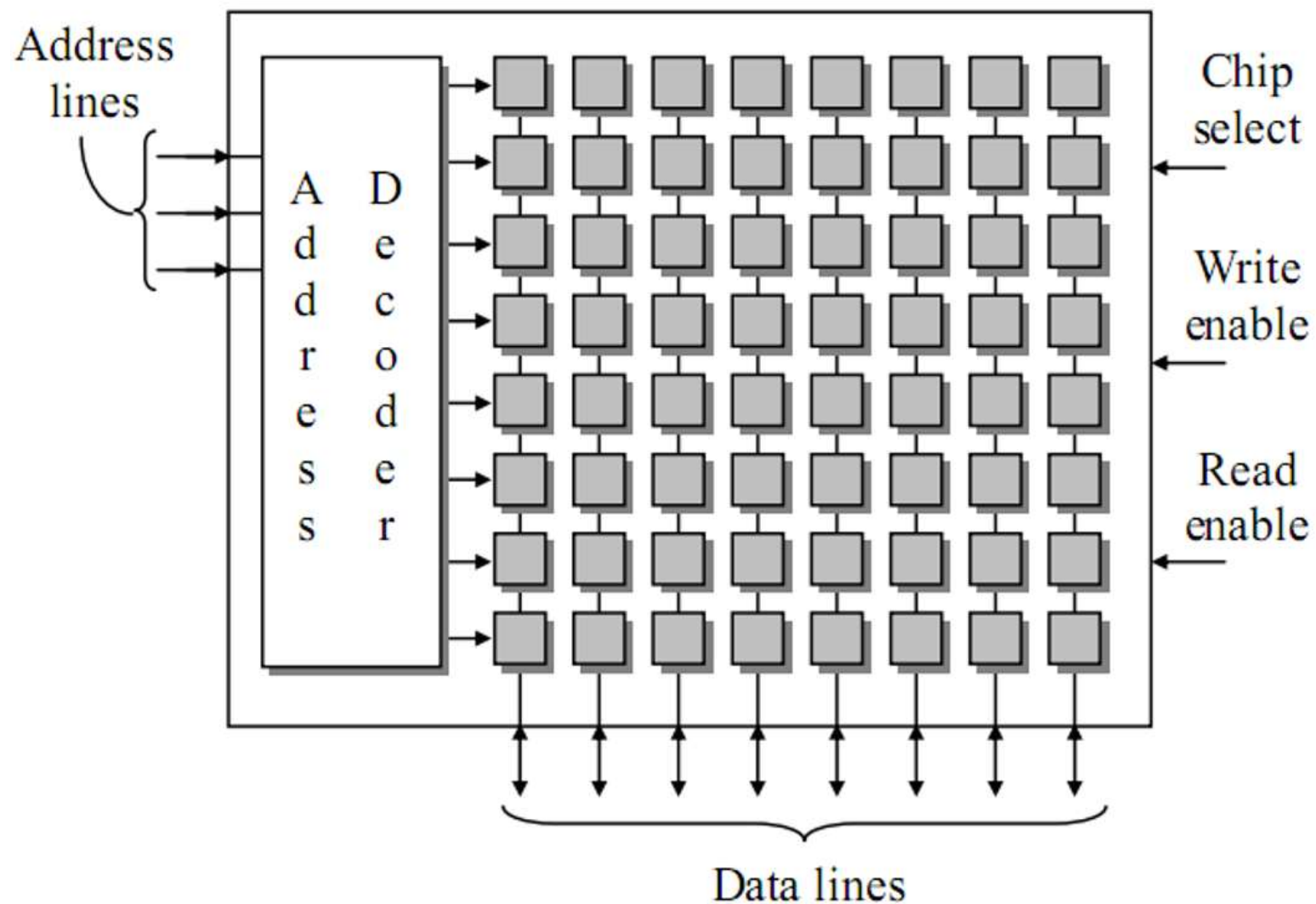
Một ô nhớ SRAM loại 6T

❖ DRAM: giá thành rẻ hơn, cần mạch làm tươi



- ❖ Bus hệ thống
 - Address Bus → chọn ô nhớ
 - Data Bus → truyền dữ liệu
 - Control Bus → read/write, interrupt
- ❖ Khi MCU thiếu RAM/Flash: sử dụng SRAM ngoài/Flash ngoài
 - Cần:
 - giải mã địa chỉ,
 - tín hiệu chip select,
 - timing phù hợp

Tổ chức mạch nhớ



❖ Address lines:

- Các đường địa chỉ nối tới bus A
- Truyền tín hiệu địa chỉ từ CPU tới mạch nhớ

❖ Address decoder:

- Bộ giải mã địa chỉ
- Dùng địa chỉ để chọn và kích hoạt ô nhớ/dòng nhớ cần truy nhập

❖ Data lines:

- Các đường dữ liệu kết nối với bus D
- Truyền dữ liệu từ bộ nhớ về CPU và ngược lại

❖ Chip select CS:

- Chân tín hiệu chọn chip
- Chip nhớ được kích hoạt khi $CS=0$. Thông thường CPU chỉ làm việc với 1 chip nhớ tại 1 thời điểm

❖ Write enable WE:

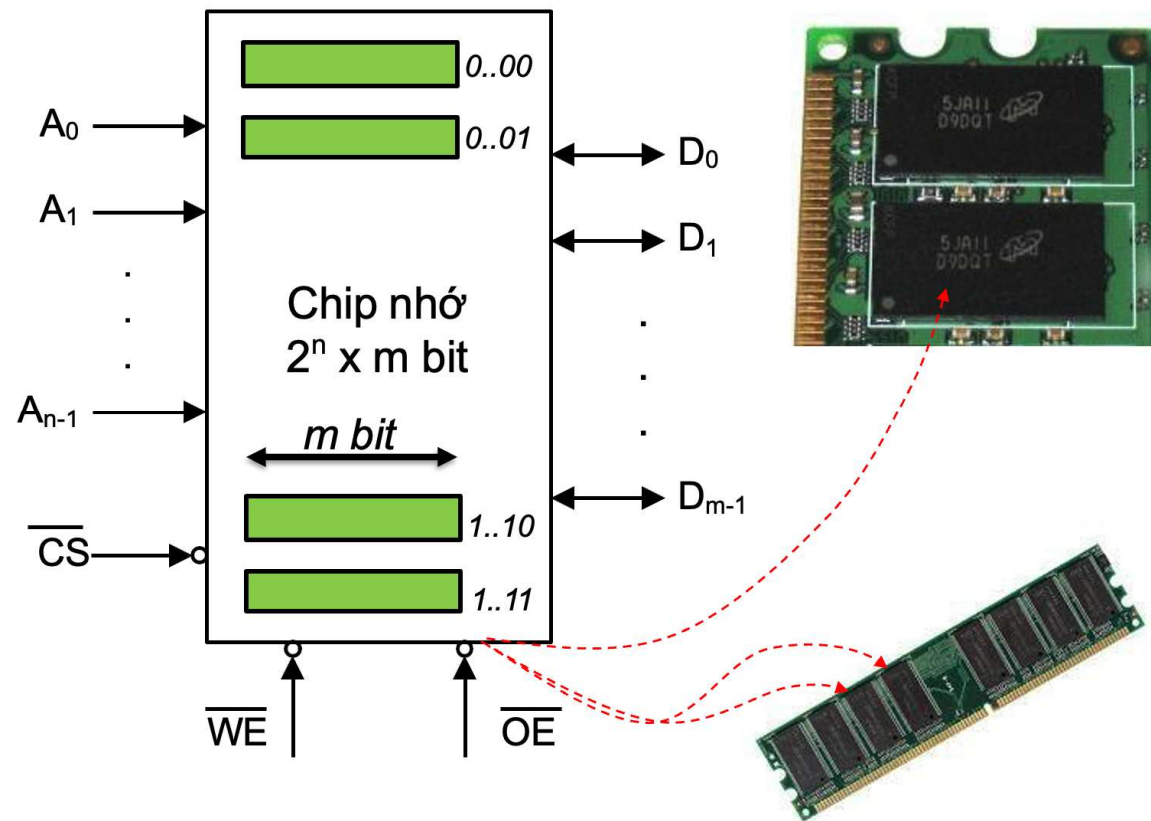
- Chân tín hiệu cho phép ghi
- Cho phép ghi vào đường nhớ khi $WE = 0$

❖ Read enable RE:

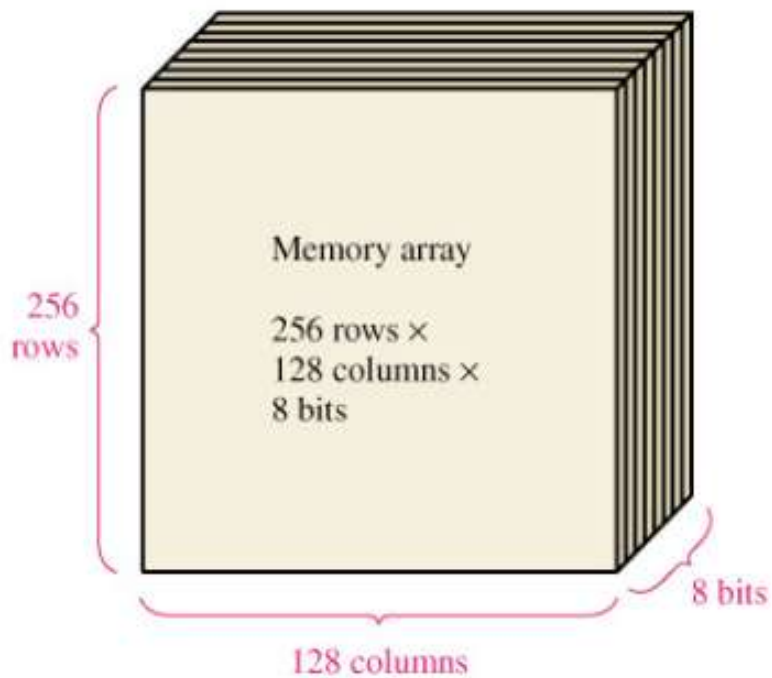
- Chân tín hiệu cho phép đọc
- Cho phép đọc dữ liệu từ đường nhớ khi $RE = 0$

❖ Basic diagram of a memory chip:

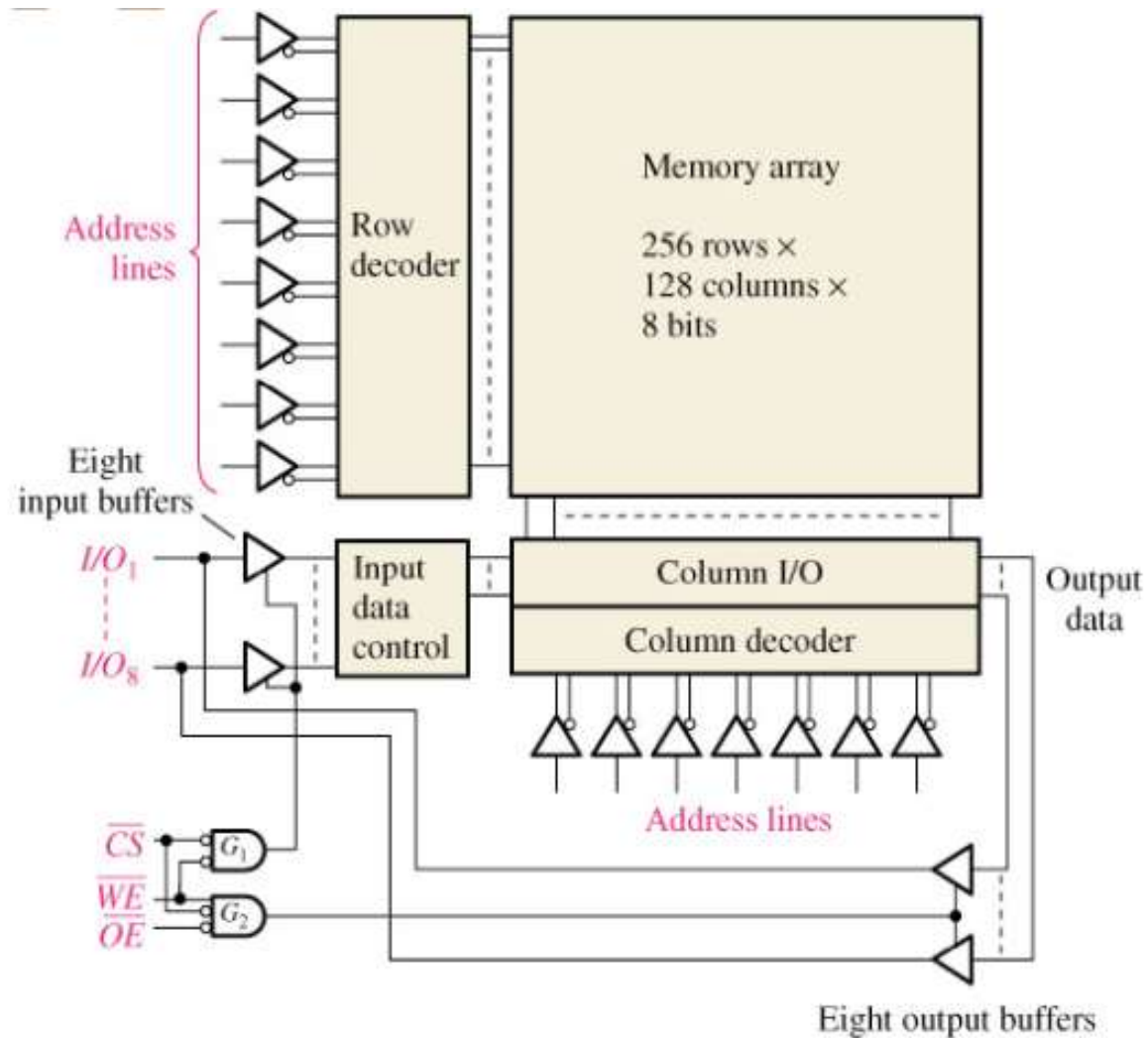
- Address lines: $A_{n-1} - A_0$
 $\rightarrow 2^n$ memory words
- Data lines: $D_{m-1} - D_0$
 $\rightarrow$ memory word length = m bits
- Memory chip capacity
 $\rightarrow 2^n \times m$ bits



Tổ chức mạch nhớ



(a) Memory array configuration



(b) Memory block diagram

❖ Thiết kế bộ nhớ

Quy trình thiết kế bộ nhớ vật lý bao gồm việc kết hợp các chip nhớ rồi để tạo thành dung lượng và độ dài từ nhớ theo yêu cầu của CPU.

- Chip nhớ là một vi mạch cụ thể, được bố trí các chân cơ bản.
- Các chân của một chip nhớ thông thường gồm các lối vào của BUS địa chỉ, lối dữ liệu, các chân điều khiển chọn chip, ghi/đọc và các chân nguồn.

❖ Thiết kế bộ nhớ

■ Các bước thực hiện

- **Thiết kế sơ đồ nguyên lý:** Sử dụng các mạch giải mã địa chỉ (như SN74138) để xác định vùng nhớ của vi trong không gian địa chỉ của CPU.
- **2. Kết nối tín hiệu điều khiển:** Nối các đường **RD (Read)**, **WR (Write)** và hệ thống BUS dữ liệu tương ứng giữa CPU và các chip nhớ.
- **3. Phân hoạch địa chỉ:** Thiết lập bảng địa chỉ (Memory Map) để quản lý vùng nào dành cho ROM (chứa BIOS/HĐH) và vùng nào dành cho RAM (dữ liệu/ứng dụng).

❖ Thiết kế bộ nhớ

■ Quy tắc tính toán thiết kế

1. Xác định số chip nhớ, để tạo được dung lượng nhớ theo yêu cầu theo công thức:

$$M = Q/D, \text{ trong đó}$$

Q là dung lượng của bộ nhớ.

D là dung lượng của mỗi chip

M là số chip nhớ cần thiết.

2. Xác định số dây địa chỉ cơ sở (tức là số dây địa chỉ thấp được nối trực tiếp vào chip nhớ hoặc chip liên thông): theo biểu thức sau:

$$2^m = D, \text{ trong đó}$$

m là số dây của địa chỉ cơ sở.

D là dung lượng của mỗi chip

3. Xác định số dây địa chỉ tạo chọn chip theo biểu thức sau:

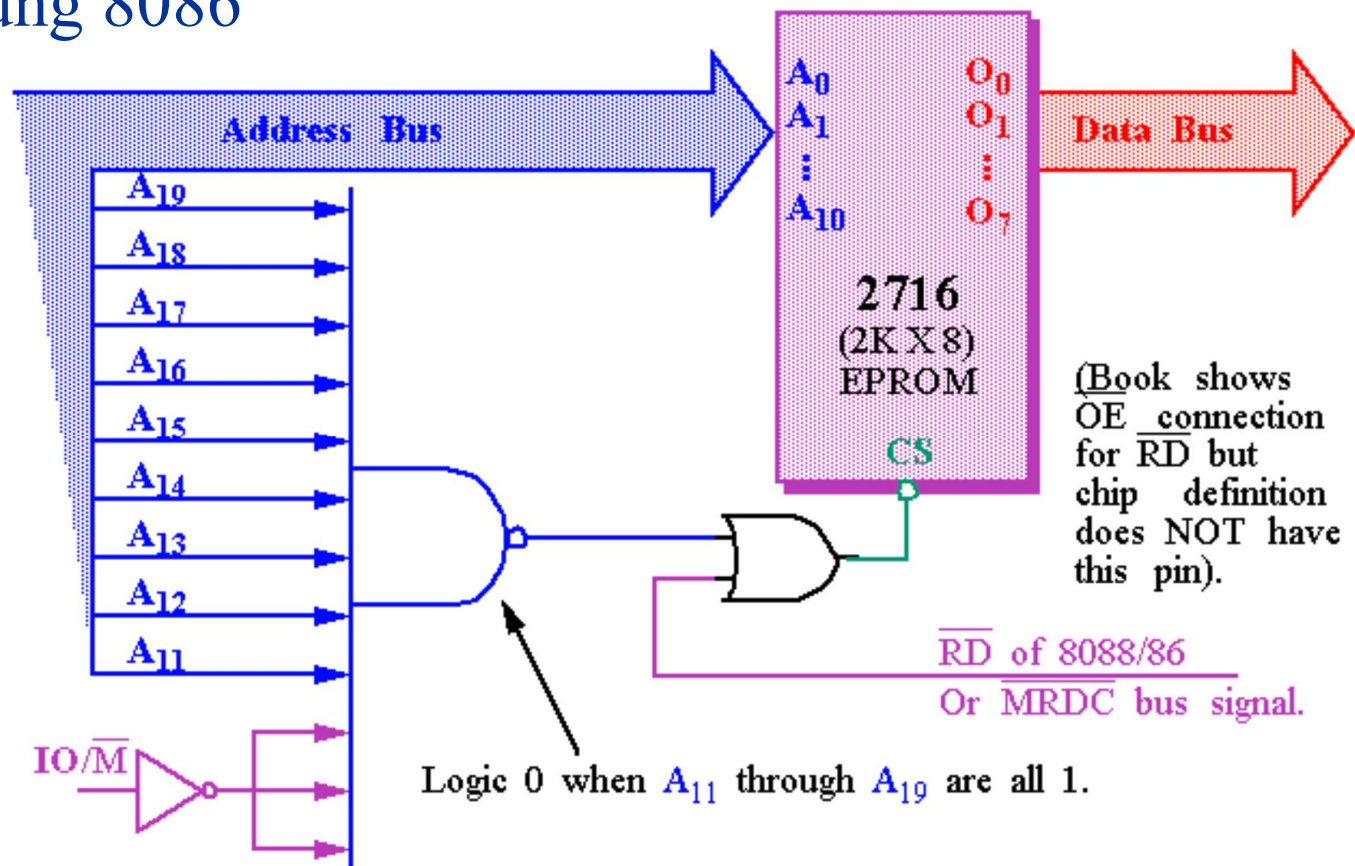
$$2^i = M, \text{ trong đó}$$

M là số lượng chip

i là số dây cần để giải mã xác định các tín hiệu chọn chip (Csi)

- ❖ Các bước giải mã địa chỉ tổng quát: Xây dựng mạch giải mã địa chỉ cho một bộ nhớ ROM. Biết rằng tổng dung lượng cần xây dựng là C_{IC} ; Kích thước mỗi vi mạch nhớ là IC, và địa chỉ cơ sở là DCCS. Giả sử dùng 8086 có 20 bit địa chỉ
 - Bước 1: Xác định số bit địa chỉ nội bộ chip và số bit giải mã.
 - Số bit địa chỉ nội bộ chip là $m = \log_2 IC \Rightarrow$ số bit dành cho giải mã mạch là $20 - m$
 - Bước 2: Phân giải địa chỉ cơ sở của các chip.
 - Số lượng chip nhớ là $M = C_{IC}/IC$ (cần thiết: $20 - m > \log_2 M$).
 - Mỗi chip nhớ có: End address = Start address + Chip capacity - 1.
 - Address of IC 1: Từ DCCS tới $(DCCS + IC - 1H)$.
 - Address of IC 2: Từ $(DCCS + IC)$ tới $(DCCS + 2 \times IC - 1H)$
 - ...
 - Bước 3: Lập bảng bit
 - Bước 4: Vẽ mạch giải mã
 - Có thể dùng các mạch logic cơ bản (AND, OR, XOR, NOT, NAND, and NOR)

- ❖ **Giải mã địa chỉ bộ nhớ sử dụng mạch logic cơ bản:** Xây dựng mạch giải mã địa chỉ cho 1 bộ nhớ ROM có dung lượng 4KB bằng phương pháp sử dụng mạch logic cơ bản;
 Biết rằng kích thước 1 vi mạch nhớ là 2Kx8 và địa chỉ cơ sở là 03800H. Giả sử dùng 8086



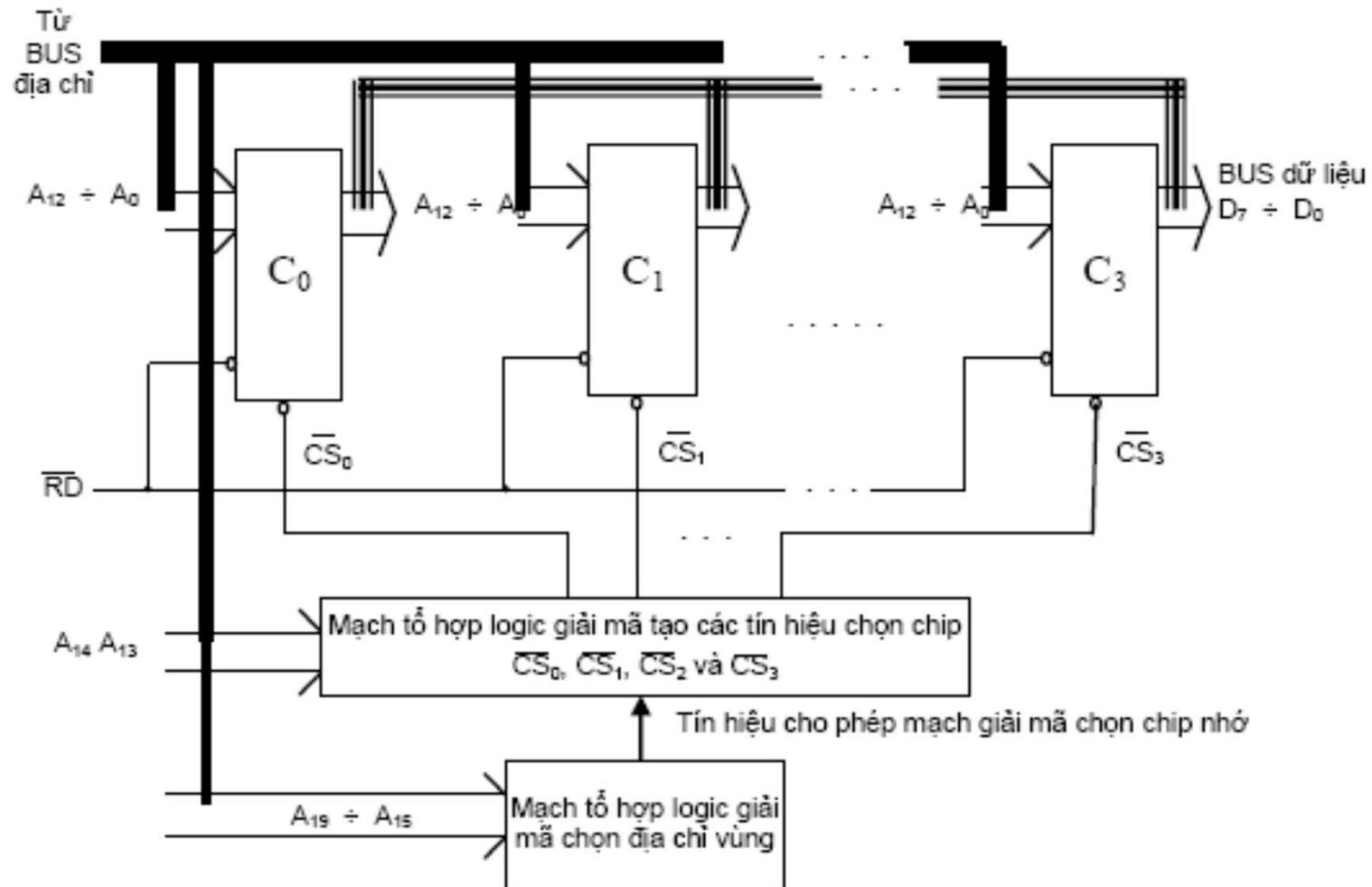
❖ **Thiết kế ROM/EPROM:** Xây dựng module ROM có dung lượng 32KB, sử dụng Chip 2764 8K x 8 bit, địa chỉ đầu là 20000hex.

- CPU x86 sử dụng 20 dây địa chỉ cho 1 MB đầu tiên: A19 – A0:
- Để truy nhập vào 8 K địa chỉ /chip cần 13 đường địa chỉ, từ A12-A0
- Để truy nhập vào mỗi chip trong 4 chip cần 2 dây địa chỉ: A14-A13

Chip #	Địa chỉ đầu	Địa chỉ cuối	Dung lượng
C0	20000	21FFF	8 K
C1	22000	23FFF	8 K
C2	24000	25FFF	8 K
C3	26000	27FFF	8 K

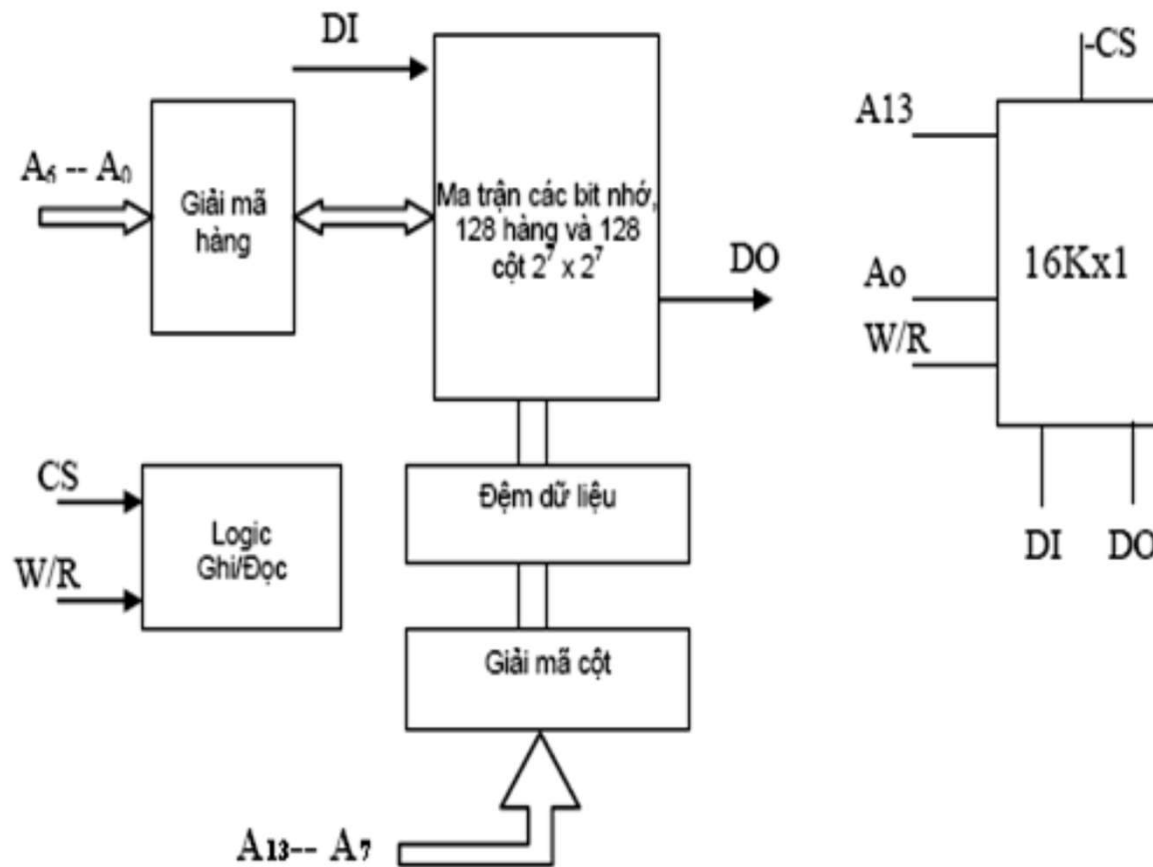
- Để xác định vùng địa chỉ cần 5 đường địa chỉ: A14-A19 của 1MB đầu tiên

- ❖ Thiết kế ROM/EPROM: Xây dựng module ROM có dung lượng 32KB, sử dụng Chip 2764 8K x 8 bit, địa chỉ đầu là 20000hex.

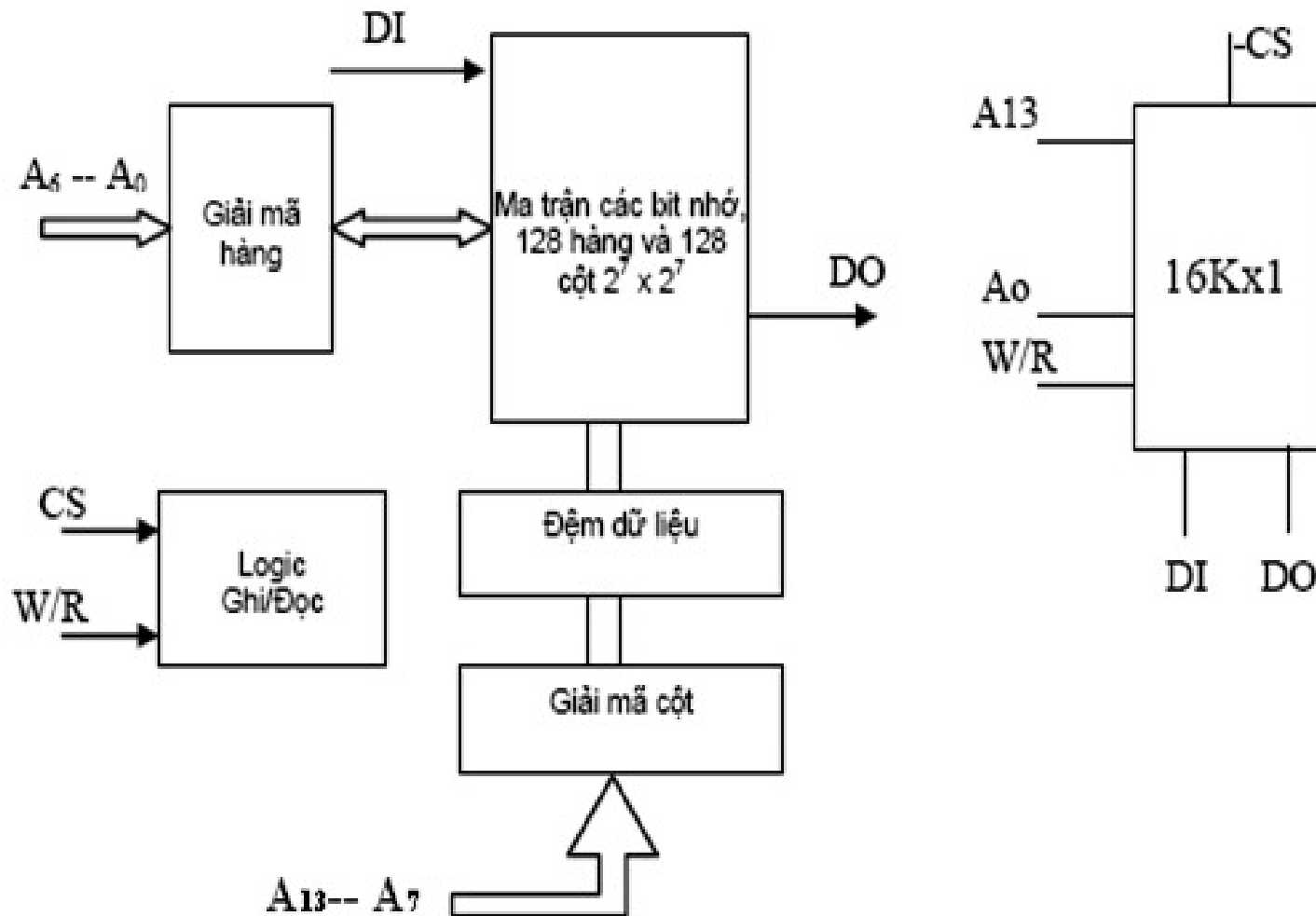


- ❖ Thiết kế RAM tĩnh (Static RAM): Xây dựng một bộ nhớ kích thước 16Kbyte trên cơ sở các chip SRAM loại 16Kx1bit. Bảng nhớ SRAM 16Kbyte được xây dựng trên cơ sở 8 chip SRAM loại 16K x 1bit, để có được ô nhớ có độ dài 8 bits.
 - Để làm được điều này người ta sắp đặt 8 chip SRAM loại 16K x 1bit sao cho mỗi chip tại một vị trí xác định sẽ đảm nhiệm lưu trữ bit dữ liệu có trọng số tương ứng trong byte dữ liệu.
 - Các đường tín hiệu:
 - A13 - A0 BUS địa chỉ, đủ cho 16K (16383) địa chỉ
 - CS: Tín hiệu chọn chip. Nếu CS = 0 thì truy nhập được chip
 - W/R: Tín hiệu điều khiển ghi/đọc. W=0 (W/) điều khiển ghi, đồng thời điều khiển các vi mạch 3 trạng thái theo các chiều In/Out
 - Chọn bảng nhớ cần thêm A14, A14=0 chọn bảng thứ nhất, (A14=1 chọn bảng thứ 2).

- ❖ Thiết kế RAM tĩnh (Static RAM): Xây dựng một bộ nhớ kích thước 16Kbyte trên cơ sở các chip SRAM loại 16Kx1bit. Bảng nhớ SRAM 16Kbyte được xây dựng trên cơ sở 8 chip SRAM loại 16K x 1bit, để có được ô nhớ có độ dài 8 bits.



❖ Thiết kế RAM tĩnh (Static RAM): loại 16K x 1bit

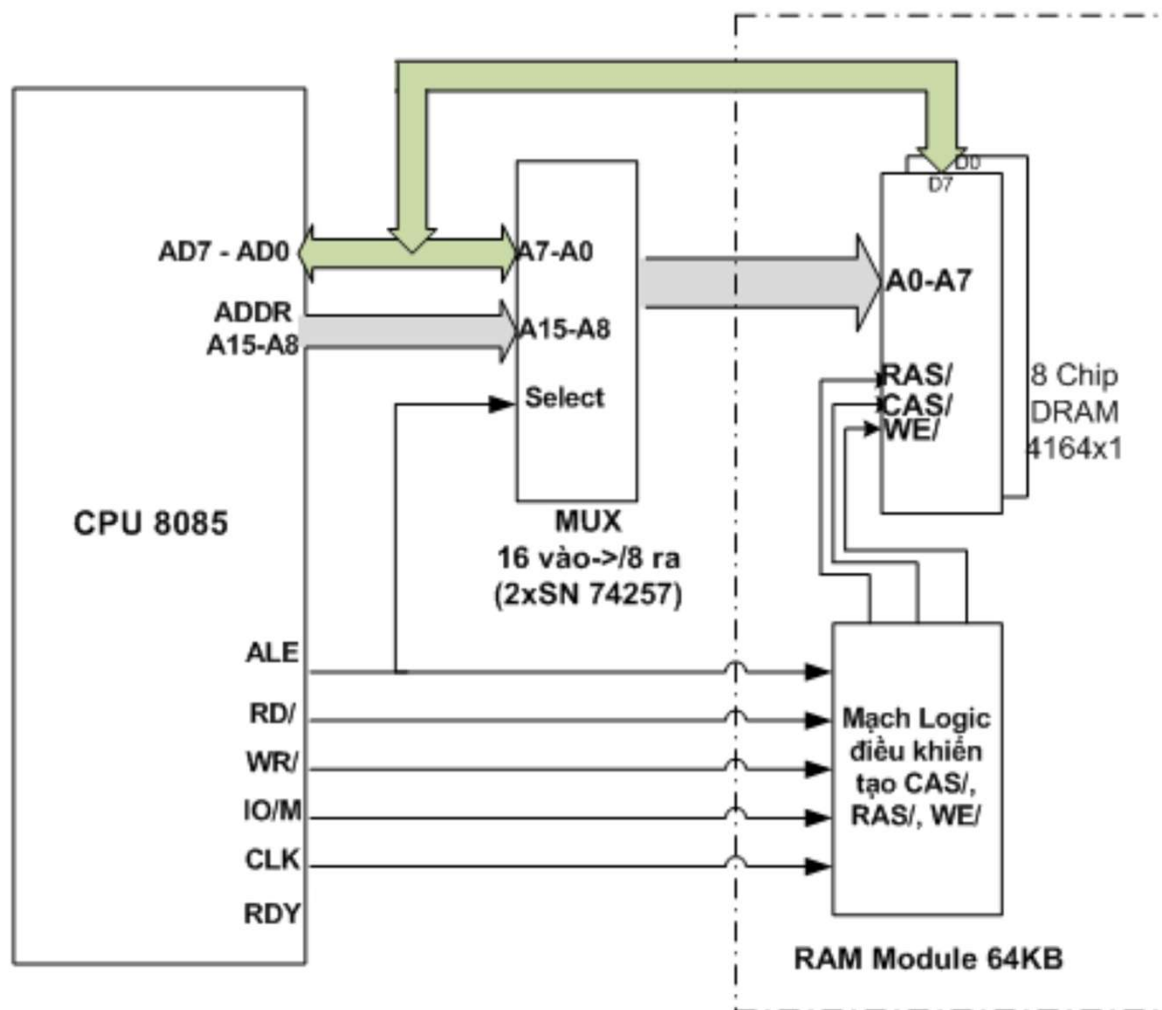


❖ Thiết kế RAM động (DRAM): DRAM MK 4164 là DRAM 64K bit trong 1 Chip. Giả định sẽ thiết kế RAM cho CPU 8085 với RAM tối đa 64 KB.

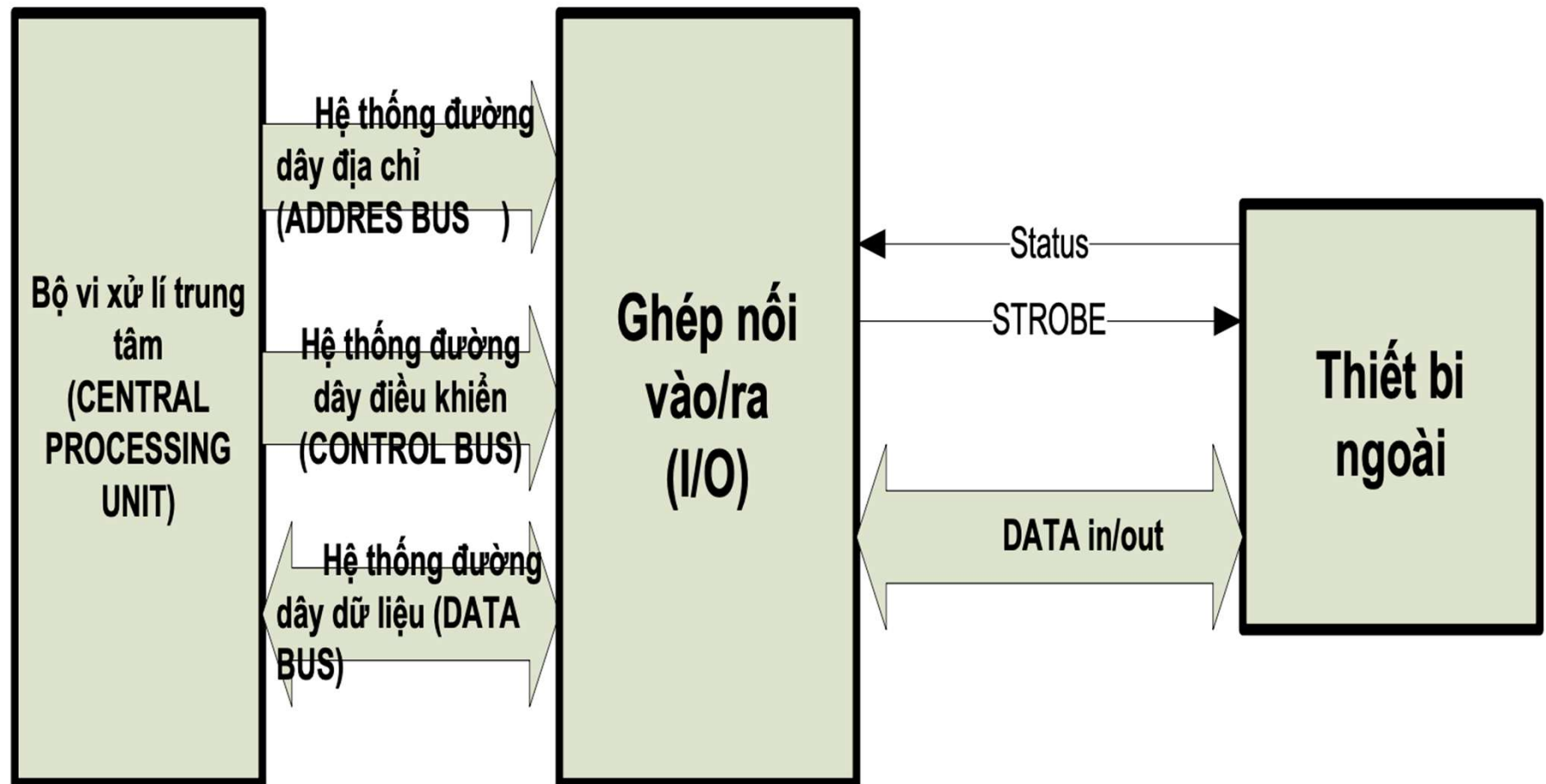
- Cần 8 chip
- DRAM 4164 có dung lượng 64 Kbit/chip, được tổ chức kiểu ma trận gồm (256 hàng x 256 cột).
- Để chọn một bit trong ma trận cần có địa chỉ ở vị trí hàng và cột xác định bởi hai bộ giải mã.
 - Đầu vào cho mỗi bộ giải mã cần 8 bit để có 256 đầu ra: A7-A0 cho hàng và A15-A8 cho cột.
 - CPU phát địa chỉ vào vi mạch dồn kênh SN 74257: A7-A0 chốt vào DRAM bằng xung RAS; A15-A8 bằng xung CAS/.
 - Khi thiết kế mạch logic điều khiển tạo RAS, CAS, và WE cần sử dụng các tín hiệu điều khiển từ CPU phát ra.

Thiết kế bộ nhớ

- ❖ Thiết kế RAM động (DRAM): DRAM MK 4164 là DRAM 64K bit trong 1 Chip. Giả định sẽ thiết kế RAM cho CPU 8085 với RAM tối đa 64 KB.



❖ Mô hình kỹ thuật ghép nối



❖ Một mô hình ghép nối điển hình trong hệ thống nhúng thường bao gồm ba thành phần cốt lõi.

- Hệ thống BUS

- BUS địa chỉ: Xác định ngoại vi nào đang được CPU truy cập.
- BUS dữ liệu: Truyền dữ liệu hai chiều giữa CPU và ngoại vi.
- BUS điều khiển: Mang các tín hiệu như Read, Write, Enable... để điều phối hoạt động.

- Mạch ghép nối vào/ra (Interface / Ports): lớp trung gian giữa BUS của CPU và thiết bị ngoại vi.

- Mạch chốt (Latch): Giữ ổn định dữ liệu hoặc địa chỉ.
- Mạch đệm 3 trạng thái (3-state buffer): Cho phép nhiều thiết bị cùng chia sẻ BUS mà không xung đột.
- Mạch giải mã địa chỉ: Tạo tín hiệu chọn cổng, chọn thiết bị (Chip Select, Port Select).

- Thiết bị ngoại vi: Cung cấp dữ liệu cho CPU xử lý, Hoặc nhận lệnh điều khiển từ CPU để tác động ra bên ngoài.

Ghép nối với thiết bị ngoại vi

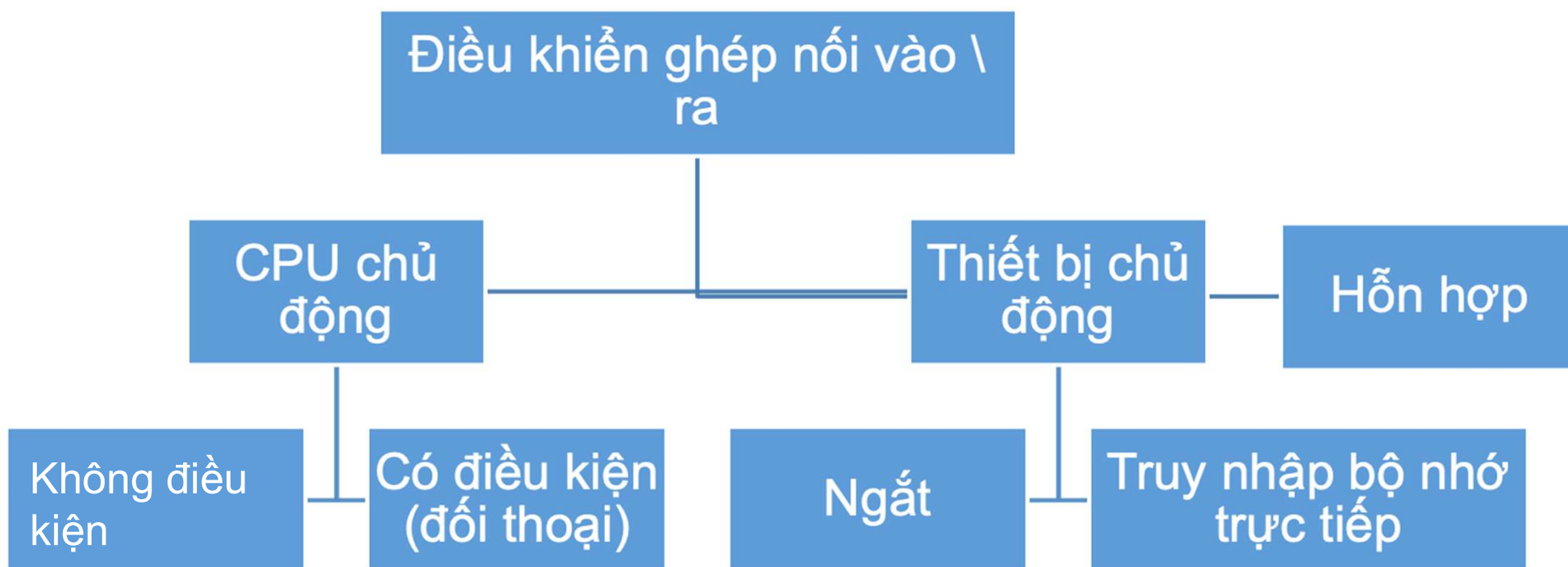
❖ Các CPU thông thường có 2 phương thức sử dụng địa chỉ

- Dùng địa chỉ để truy nhập vào bộ nhớ, thông qua các lệnh máy liên quan tới bộ nhớ

Thiết bị	Phân bổ địa chỉ	Kích thước
RAM	0000 - 7FFF	RAM 32 KB
Cho các cổng sẽ dùng (I/O ports)	8000 - 80FF	256 bytes
Sound controller (I/O ports)	9000 - 90FF	256 bytes
Video controller/text-mapped display RAM	A000 - A7FF	2KB
ROM	C000 - FFFF	16 KB

- Dùng một số rất ít lệnh đặc biệt để truy nhập vào các thiết bị (vi mạch khả trình) để gọi các mã điều khiển tới các thiết bị đó.
 - CPU truy nhập vào thiết bị theo cổng (port).
 - Ví dụ lệnh: IN des, src; OUT des, src
 - *src* thường là DX chứa [địa_chỉ_port_in] hay [địa_chỉ_port_out]
 - *des* thường là AL hay AX chứa dữ liệu đọc vào hay ghi ra

❖ Các kiểu ghép nối



❖ Các kiểu ghép nối

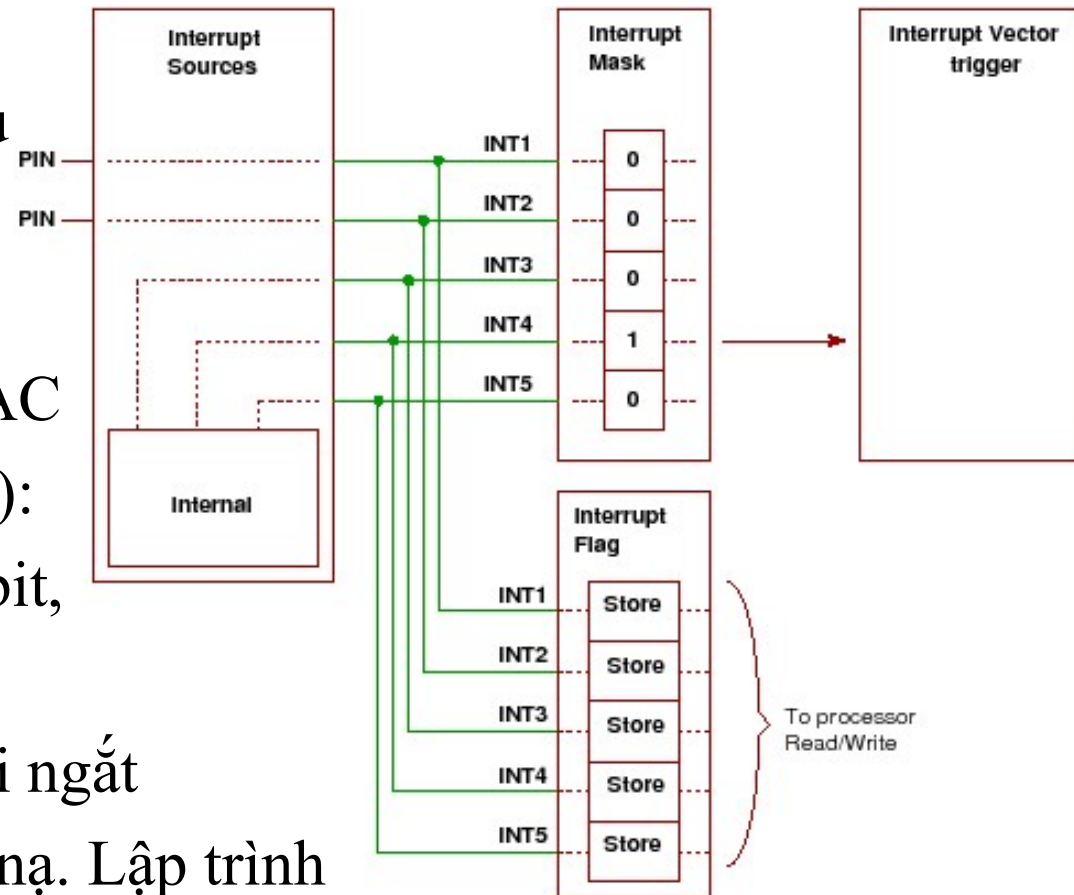
- Ghép nối do CPU chủ động (CPU-active): mọi thao tác đều do CPU quyết định.
 - Ghép nối không điều kiện: CPU giả định thiết bị luôn sẵn sàng, không kiểm tra trạng thái. Phù hợp với các thiết bị rất đơn giản (LED, Công tắc, Rơ-le)
 - Ưu điểm: Mạch và phần mềm đơn giản
 - Nhược điểm: Không an toàn với thiết bị chậm
- Ghép nối có điều kiện (Handshaking): CPU phải kiểm tra trạng thái của thiết bị trước khi trao đổi dữ liệu thông qua một cổng trạng thái (Status Port).
 - Nếu thiết bị chưa sẵn sàng ($READY = 0$), CPU sẽ: Chờ Hoặc kiểm tra lại sau
 - Kiểu này đảm bảo đồng bộ chính xác, tránh mất dữ liệu.
- Quay vòng – Polling: CPU lần lượt kiểm tra trạng thái của nhiều thiết bị ngoại vi theo một thứ tự xác định.
 - Nếu thiết bị nào sẵn sàng $\rightarrow$ CPU phục vụ ngay. Sau đó tiếp tục kiểm tra thiết bị khác
 - Dễ cài đặt, kiểm soát nhưng Lãng phí thời gian xử lý nếu ít thiết bị hoạt động

❖ Các kiểu ghép nối

- Ghép nối do thiết bị chủ động (I/O-active): thiết bị ngoại vi mới là bên chủ động, CPU chỉ phản hồi khi cần.
 - Ngắt (Interrupt)
 - Khi thiết bị cần phục vụ: Nó gửi tín hiệu IRQ tới CPU
 - CPU: Tạm dừng chương trình chính, Lưu trạng thái, Nhảy tới chương trình phục vụ ngắt (ISR), Xử lý xong rồi quay lại công việc cũ
 - Ưu điểm: CPU không phải chờ đợi, Hiệu quả sử dụng CPU cao, Phù hợp với hệ thống thời gian thực. Đây là phương pháp rất phổ biến trong vi điều khiển và RTOS.
 - Truy nhập bộ nhớ trực tiếp – DMA: CPU không trực tiếp truyền dữ liệu. Thay vào đó, bộ điều khiển DMA (DMAC) sẽ: Nhận quyền điều khiển BUS, Truyền dữ liệu trực tiếp giữa RAM và thiết bị.
 - Trong thời gian này: CPU tạm thời “nhường BUS”, Nhưng vẫn có thể xử lý các tác vụ khác
 - DMA giúp: Tăng tốc độ truyền dữ liệu, Giảm tải cho CPU

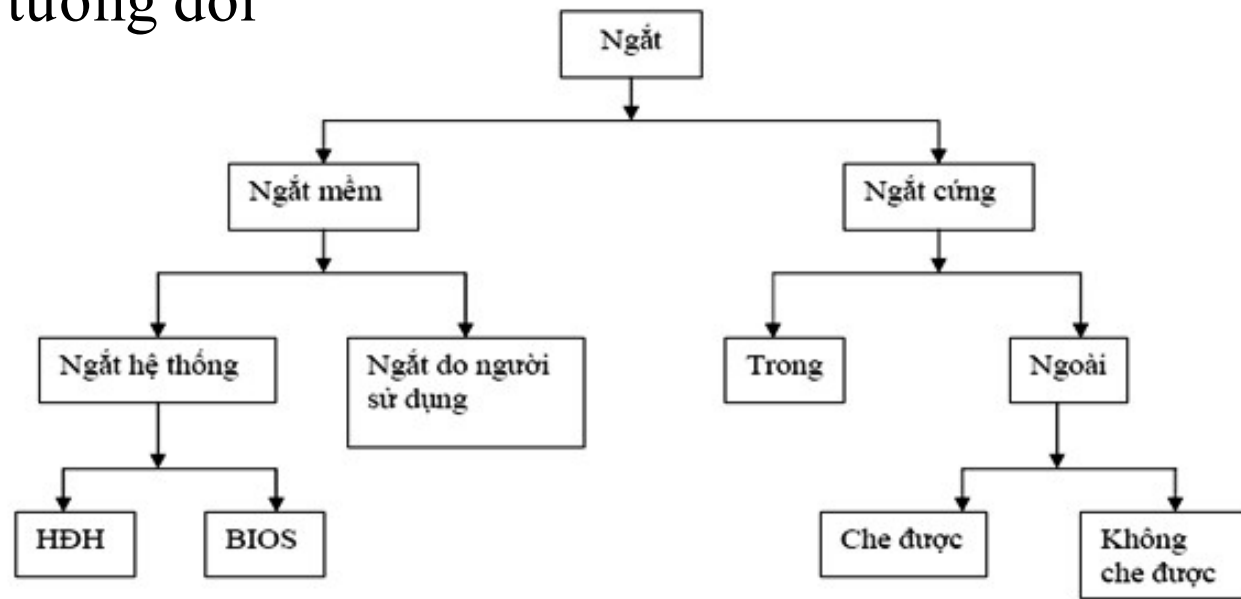
❖ Một hệ ngắt cơ bản:

- Nguồn phát sinh ngắt: các tín hiệu ngắt từ các thiết bị trong hay ngoài bo mạch, ví dụ từ ADC/DAC, từ UART, từ các cổng ghép nối, DMAC
- Cờ trạng thái ngắt (Interrupt flags): Mỗi ngắt kết hợp với cờ, chỉ cần 1 bit, báo trạng thái của ngắt đó.
- Mặt nạ ngắt (Interrupt mask): Mỗi ngắt kết hợp với 1 bit của thanh ghi mặt nạ. Lập trình các bit của thanh ghi sẽ cho phép hay cấm ngắt tương ứng kích hoạt.



❖ Phân loại kiểu ngắt:

- Ngắt mềm thực chất thực hiện một lời gọi hàm đặc biệt được kích hoạt bởi các nguồn ngắt là các sự kiện xuất hiện từ bên trong chương trình và ngoại vi tích hợp trên Chip. Đây là cơ chế ngắt đồng bộ với chương trình.
- Ngắt cứng có thể được xem như là một lời gọi hàm đặc biệt trong đó nguồn kích hoạt là một sự kiện đến từ bên ngoài chương trình thông qua một cấu trúc phần cứng. Đây là cơ chế dị bộ vì các sự kiện ngắt kích hoạt từ các tín hiệu ngoại vi bên ngoài và tương đối độc lập với CPU.



❖ Các phương thức ghép nối cơ bản

- GPIO (Digital I/O)
 - Cách hoạt động:
 - HIGH (1) $\rightarrow$ 3.3V hoặc 5V,
 - LOW (0) $\rightarrow$ 0V
 - Ứng dụng: bật/tắt LED, đọc nút nhấn, điều khiển relay.
- Analog Interface
 - ADC (Analog $\rightarrow$ Digital)
Đọc tín hiệu analog: nhiệt độ, điện áp, ánh sáng
 - DAC (Digital $\rightarrow$ Analog)
Xuất tín hiệu analog: âm thanh, điều khiển thiết bị analog
- PWM (Pulse Width Modulation)
 - PWM tạo tín hiệu xung với duty cycle thay đổi.
 - Ứng dụng: điều khiển tốc độ động cơ, điều chỉnh độ sáng LED, điều khiển servo

❖ Ghép nối ngoại vi bằng BUS truyền thông:

- Khi thiết bị phức tạp, cần dùng bus truyền dữ liệu
- Các chuẩn kết nối cơ bản: UART, SPI, I2C

	UART	SPI	I2C
Đồng bộ	✗	✓	✓
Số dây	2	4+	2
Tốc độ	TB	Rất cao	TB
Nhiều thiết bị	✗	✓	✓
Địa chỉ	✗	✗	✓

- Chuẩn kết nối nâng cao: CAN, USB, Ethernet

❖ Ghép nối ngoại vi bằng BUS truyền thông:

- UART (Universal Asynchronous Receiver/Transmitter)
 - Đặc điểm:
 - giao tiếp nối tiếp không đồng bộ,
 - không cần clock chung,
 - cấu hình baud rate
 - Dây kết nối: TX (truyền), RX (nhận), GND
 - Khung dữ liệu: START | DATA | PARITY | STOP
 - Ưu điểm: đơn giản, dễ debug
 - Nhược điểm: chỉ point-to-point, tốc độ trung bình
 - Ứng dụng: debug serial monitor, GPS module, Bluetooth module
 - Chọn UART khi:
 - cần giao tiếp PC,
 - debug hệ thống,
 - cấu hình đơn giản

❖ Ghép nối ngoại vi bằng BUS truyền thông:

▪ SPI (Serial Peripheral Interface)

• Đặc điểm

- giao tiếp đồng bộ
- tốc độ rất cao
- full duplex

• Dây kết nối

Dây	Chức năng
MOSI	master → slave
MISO	slave → master
SCLK	clock
SS/CS	chọn thiết bị

- Ưu điểm: rất nhanh, truyền hai chiều, ổn định
- Nhược điểm: nhiều dây, mỗi slave cần 1 chân SS
- Ứng dụng: màn hình TFT, thẻ SD, bộ nhớ Flash ngoài, cảm biến tốc độ cao
- Chọn SPI khi: cần tốc độ cao, truyền dữ liệu lớn, màn hình, thẻ nhớ

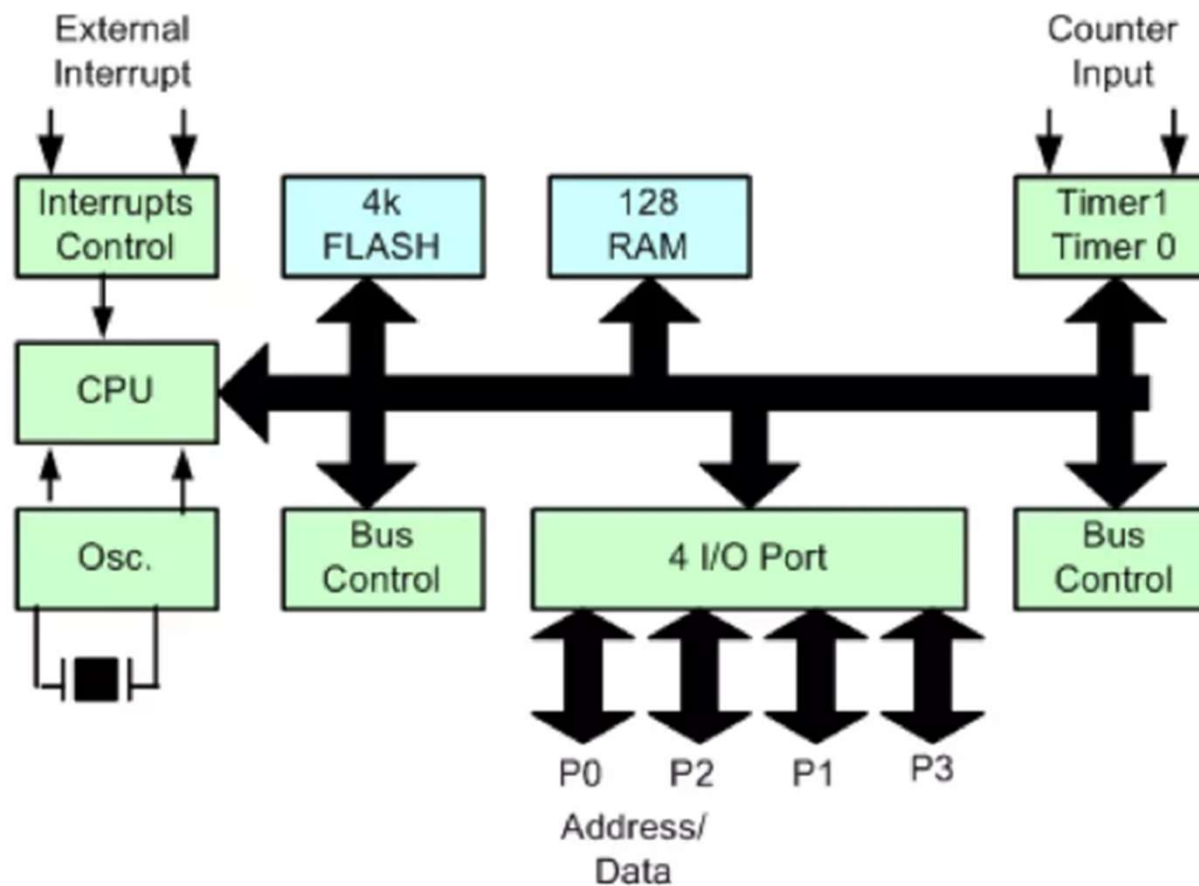
❖ Ghép nối ngoại vi bằng BUS truyền thông:

- I2C (Inter-Integrated Circuit)
- Đặc điểm:
 - nhiều thiết bị trên cùng bus,
 - mỗi thiết bị có địa chỉ
- Dây kết nối:
 - chỉ 2 dây: SDA (data), SCL (clock)
 - Cách hoạt động: Master gửi địa chỉ → slave phản hồi.
- Ưu điểm: tiết kiệm dây, hỗ trợ nhiều thiết bị, dễ mở rộng
- Nhược điểm: chậm hơn SPI, khoảng cách ngắn
- Ứng dụng: cảm biến nhiệt độ, màn hình OLED, RTC, EEPROM
- Chọn I2C khi:
 - nhiều cảm biến,
 - cần tiết kiệm dây,
 - thiết bị tốc độ thấp

❖ Ghép nối ngoại vi bằng BUS truyền thông:

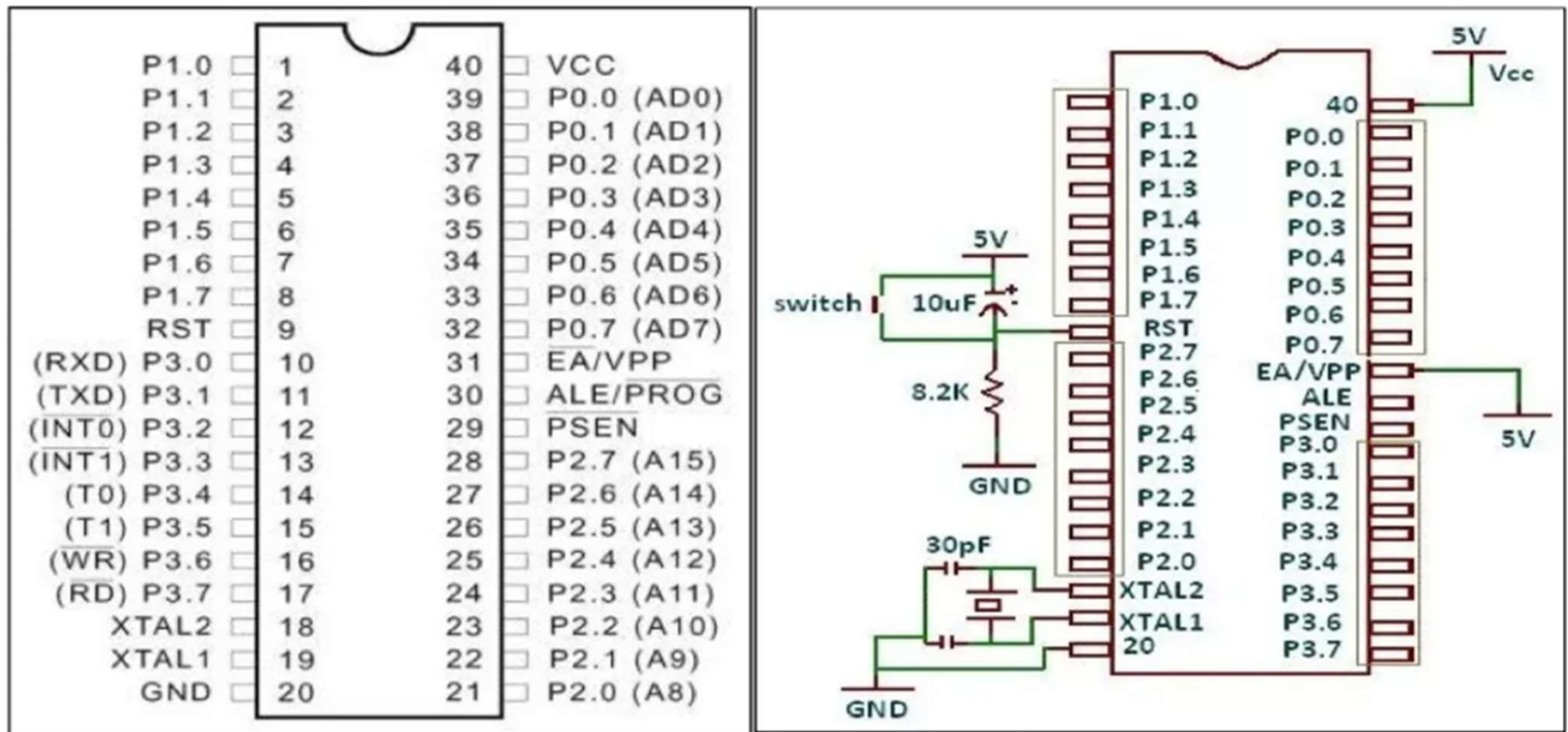
- Chuẩn kết nối nâng cao (tham khảo)
 - CAN Bus
 - dùng trong ô tô, công nghiệp
 - chống nhiễu tốt
 - USB
 - tốc độ cao
 - kết nối thiết bị ngoại vi
 - Ethernet: truyền dữ liệu mạng

❖ Sơ đồ khối chip 8051:



Vi điều khiển 8051

❖ Sơ đồ chân của 8051: 32 chân xuất/nhập, tuy nhiên có 24 chân đa mục đích trong tổng số 32 chân này.



❖ Sơ đồ chân của 8051:

- Port 0 (các chân từ 32-39) được ký hiệu là P0.0-P0.7 có hai công dụng.
 - Port 0 được sử dụng làm nhiệm vụ xuất nhập
 - Với các thiết kế lớn hơn có bộ nhớ ngoài, Port 0 trở thành bus địa chỉ và bus dữ liệu đa hợp (byte địa chỉ thấp).
- Port 1 (các chân từ 1-8) chỉ có công dụng là xuất/nhập được ký hiệu P1.0-P1.7 và dùng để giao tiếp với thiết bị bên ngoài.
- Port 2 (các chân từ 21-28) được ký hiệu là P2.0-P2.7 có hai công dụng,
 - làm nhiệm vụ xuất/nhập
 - byte địa chỉ cao của bus địa chỉ 16 bit cho các thiết kế có bộ nhớ chương trình ngoài hoặc các thiết kế có nhiều hơn 256 byte bộ nhớ dữ liệu.
- Port 3 (các chân từ 10-17) được ký hiệu là P3.0-P3.7 có hai công dụng.
 - hoạt động xuất/nhập
 - các chân của port 3 có nhiều chức năng riêng.

❖ Sơ đồ chân của 8051:

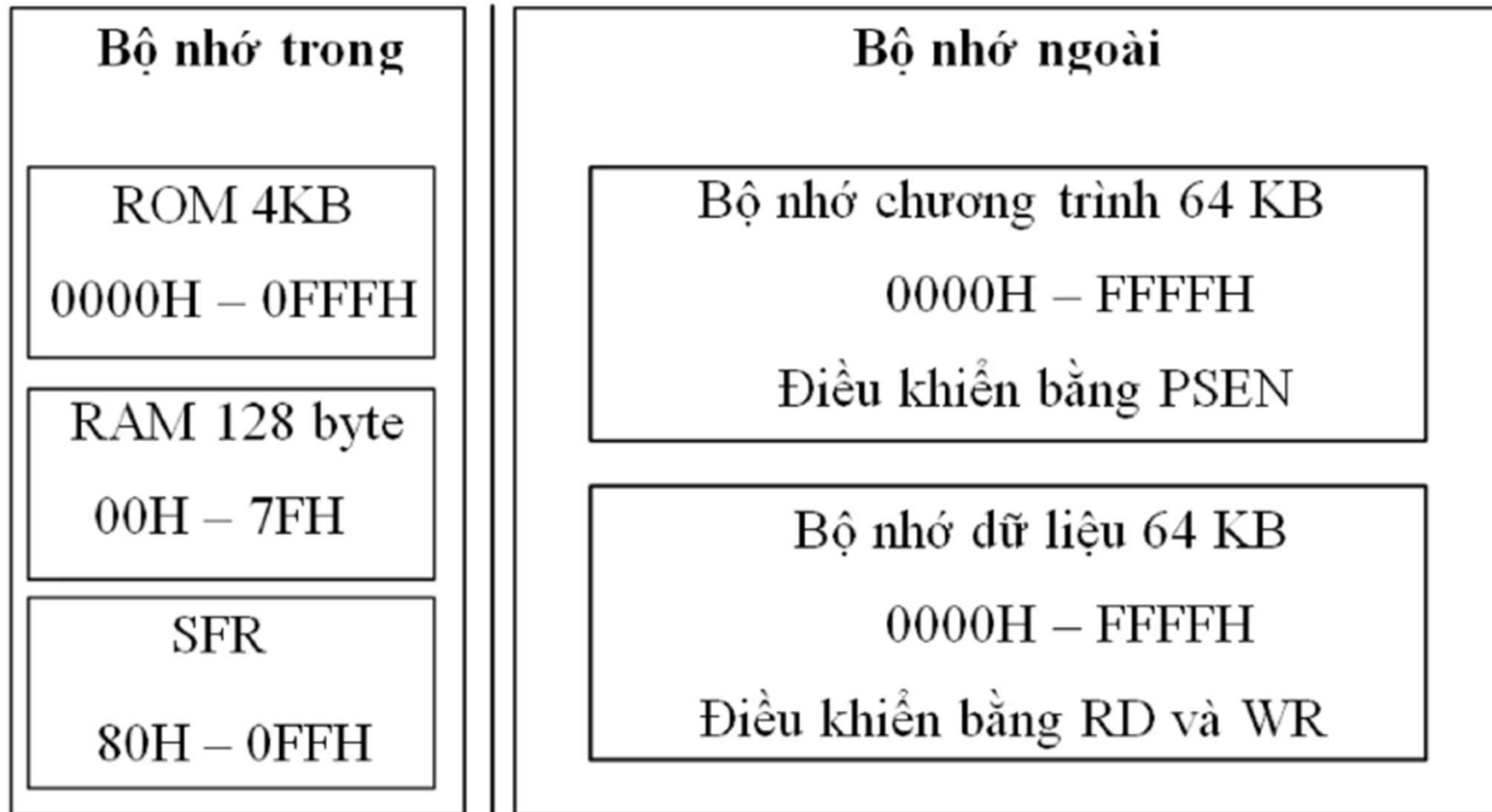
- PSEN: chân 29
 - cho phép truy xuất bộ nhớ chương trình ngoài. Tín hiệu /PSEN ở mức logic 0 trong suốt thời gian tìm nạp lệnh.
 - Khi thực thi một chương trình chứa ở ROM nội, chân /PSEN được duy trì ở mức logic không tích cực (logic 1).
- ALE: dùng để giải đa hợp (demultiplexing) bus dữ liệu và bus địa chỉ.
- EA:
 - Nếu chân này nối lên 5V chip 8051 thực thi chương trình trong ROM nội.
 - Nếu chân này được nối với GND (và chân /PSEN cũng ở logic 0) thì chương trình cần được thực thi chứa ở bộ nhớ ngoài.
- Mạch dao động trên chip được ghép nối với mạch thạch anh bên ngoài ở hai chân XTAL1 và XTAL2

❖ Sơ đồ chân của 8051:

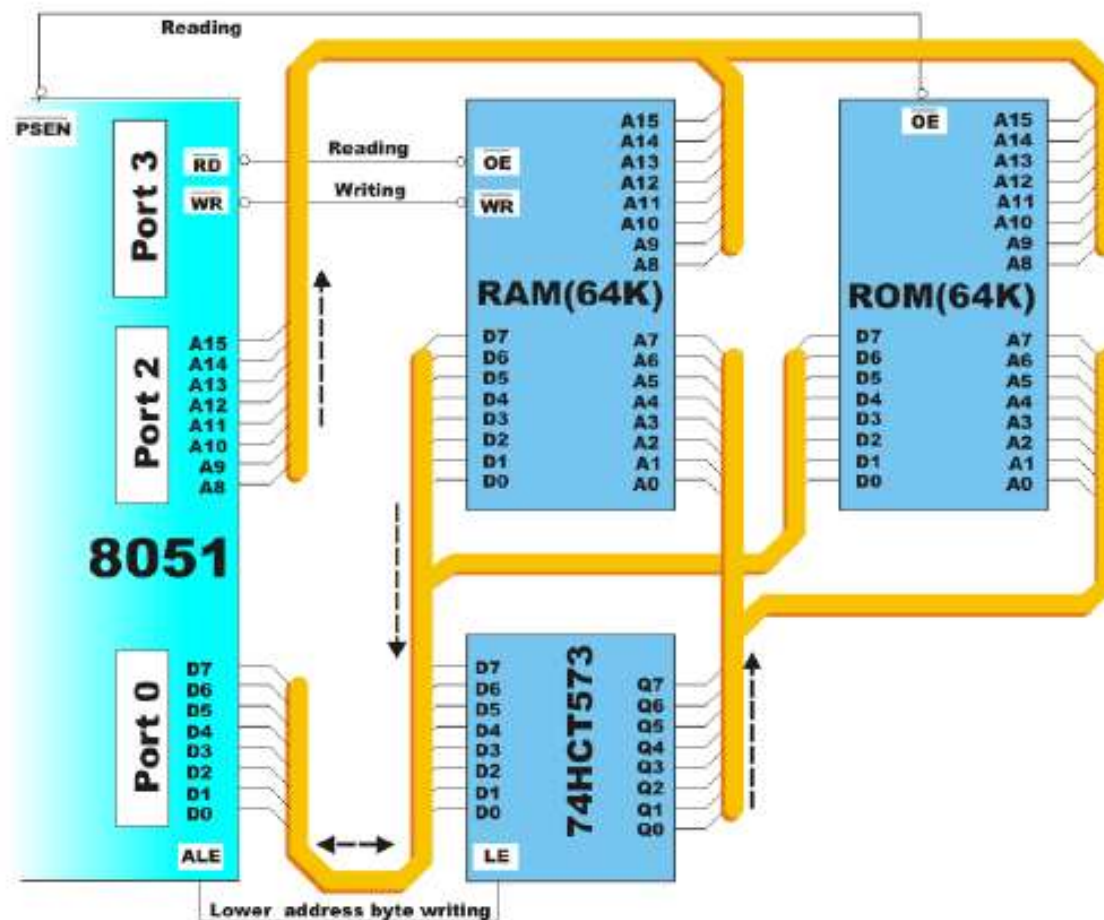
- RESET (RST): Khi ngõ vào này được treo ở mức logic 1 tối thiểu 2 chu kỳ máy, các thanh ghi bên trong của 8051 được nạp lại các giá trị thích hợp cho việc khởi động lại hệ thống.

Thanh ghi	Nội dung
Bộ đếm chương trình	0000H
Thanh chứa A	00H
Thanh ghi B	00H
PSW	00H
SP	07H
DPTR	0000H
Port 0-3	FFH
IP	xxx00000B
IE	0xx00000B
Các thanh ghi định thời	00H
SCON	00H
SBUF	00H
PCON (HMOS)	0xxxxxxxB
PCON (CMOS)	0xxx0000B

❖ Tổ chức bộ nhớ của 8051:

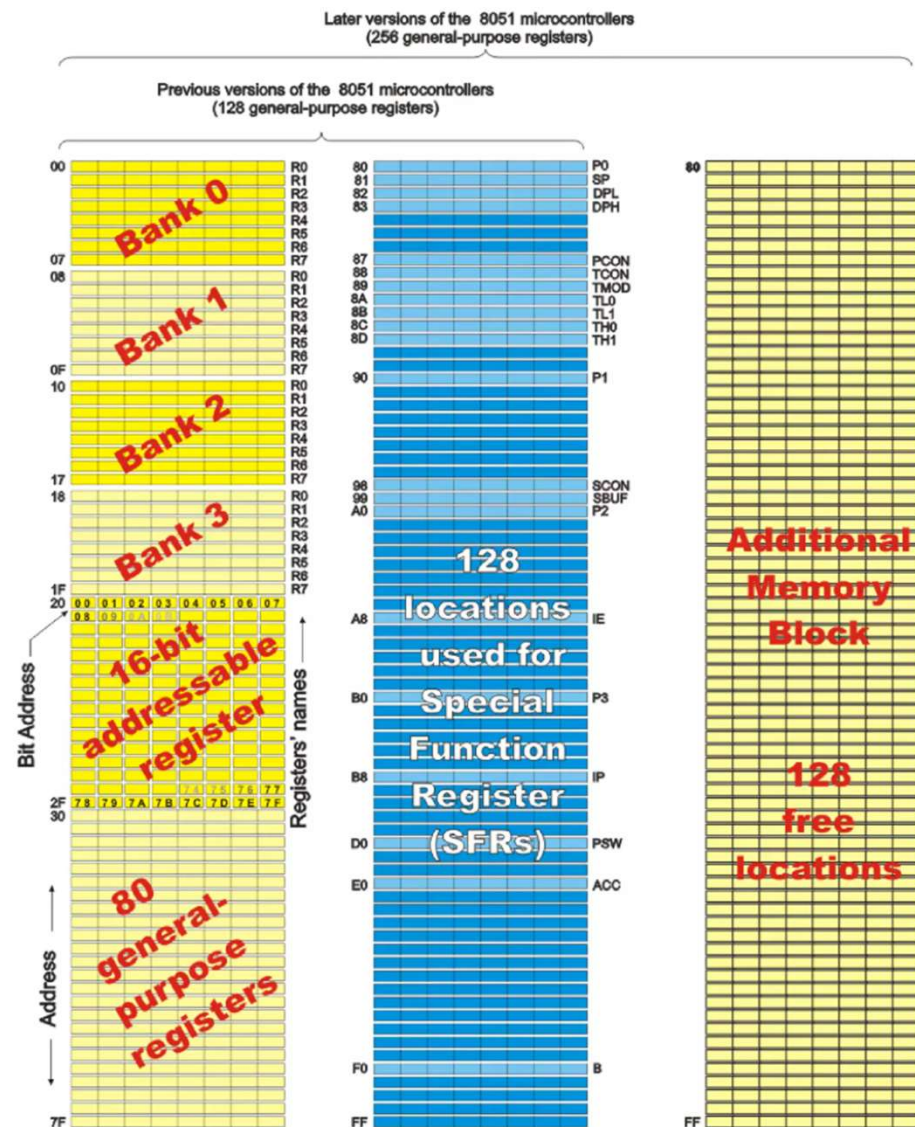


❖ Ví dụ thiết kế mở rộng bộ nhớ:

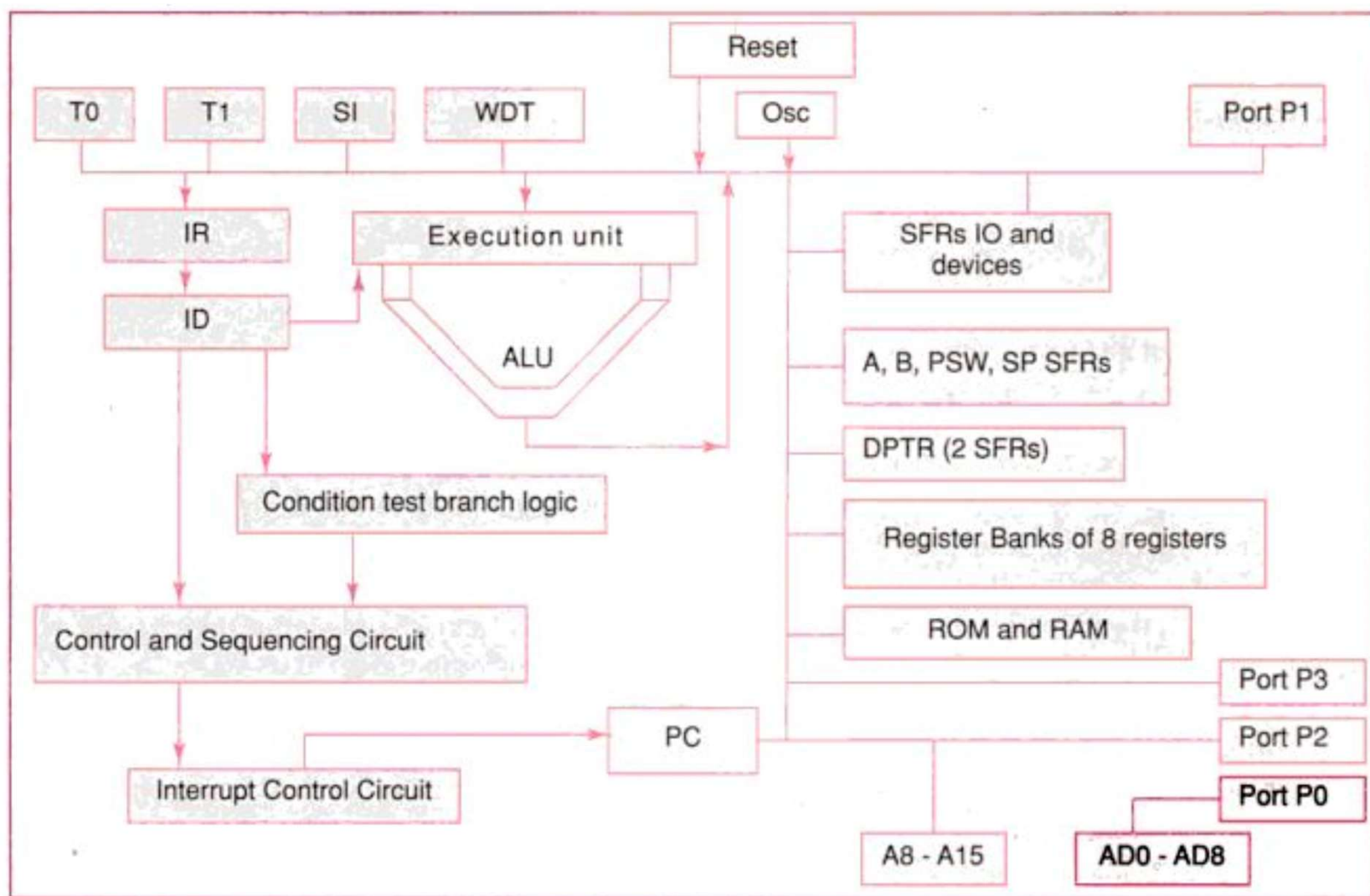


❖ Sơ đồ phân bố RAM và các thanh ghi đặc biệt:

- 21 thanh ghi chức năng đặc biệt SFR chiếm phần trên của RAM nội từ địa chỉ 80H-FFH.
- Thanh ghi Acc, B
- Con trỏ Stack (SP)
- Con trỏ dữ liệu DPTR: dùng để truy xuất bộ nhớ ngoài
- Các thanh ghi Port:
- Các thanh ghi định thời:
- Các thanh ghi port nối tiếp
- Các thanh ghi ngắt



❖ Kiến trúc cơ bản 8051:



- ❖ Kiến trúc tập lệnh
- ❖ Các chế độ địa chỉ hóa (Addressing Modes)
- ❖ Lập trình vào ra của 8051
- ❖ Ghép nối với bộ nhớ ngoài
- ❖ Counter và Timer
- ❖ Đầu vào/Đầu ra giao tiếp dữ liệu nối tiếp
- ❖ Ngắt của 8051
- ❖ Giao tiếp sử dụng Bus hệ thống

KIT HỌC TẬP 8051 BẢN PRO

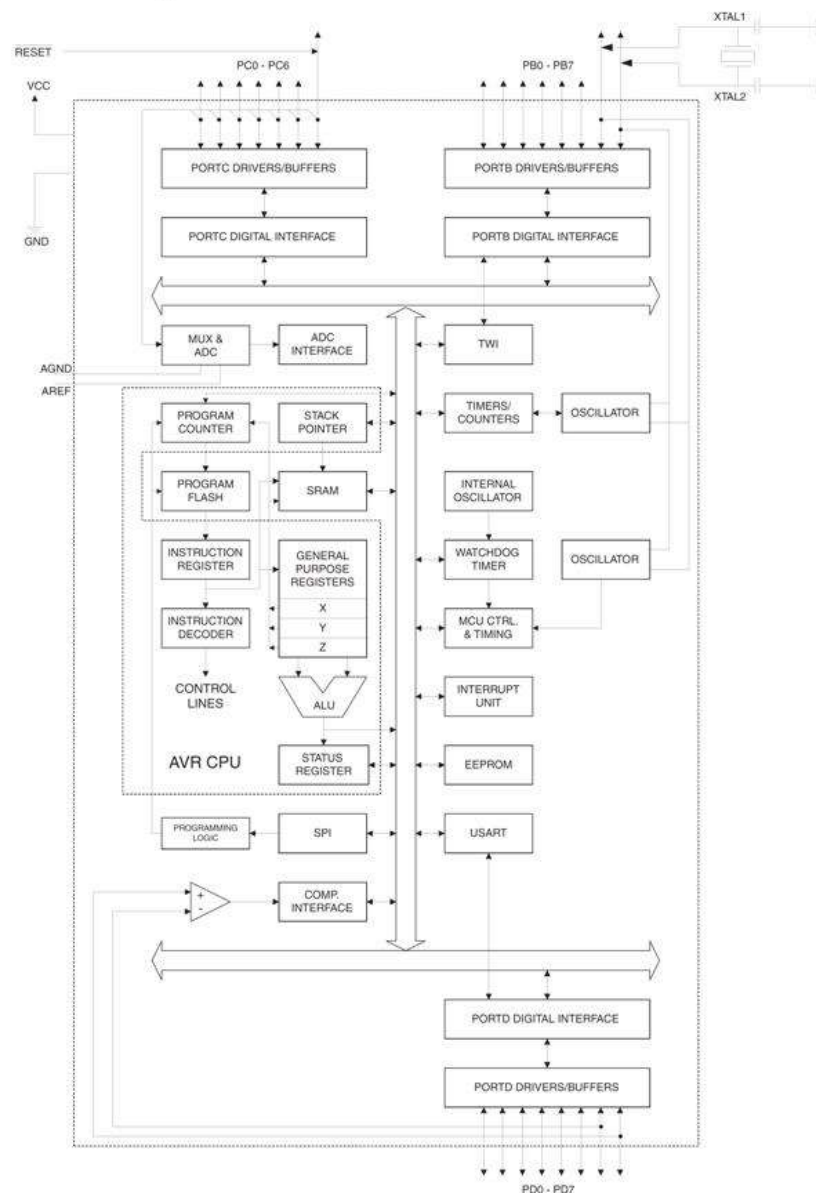
Bộ sản phẩm gồm:

- + IC 89C52RC
- + Dây usb kết nối
- + Motor DC
- + Điều khiển Remote 21
- + LCD
- + Cánh quạt
- Gói phụ kiện không bao gồm mạch nạp



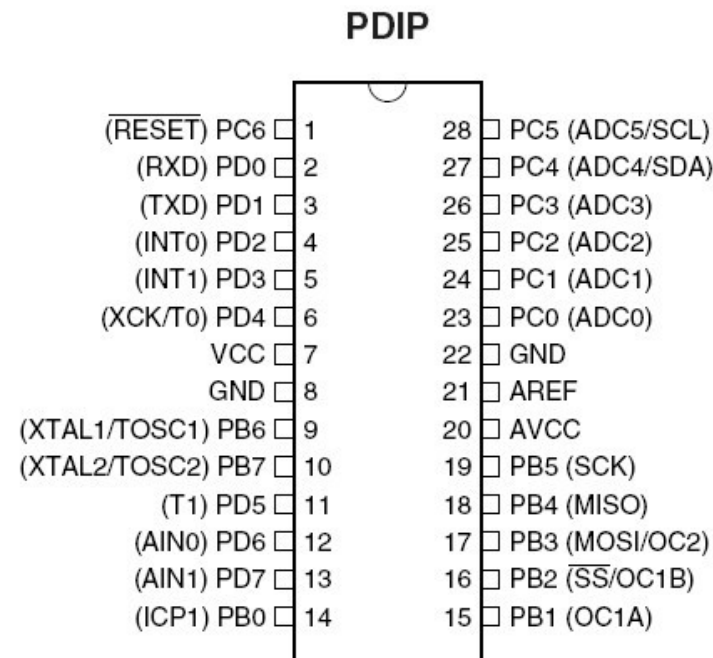
❖ Vi điều khiển Atmega8 của hãng ATMEL là một loại vi điều khiển AVR mới

- Atmega8 với kiến trúc RISC có chỉ tiêu chất lượng cao và tiêu thụ năng lượng ít
- 8K byte bộ nhớ Flash, 512 byte bộ nhớ EEPROM, 1K byte bộ nhớ SRAM
- Hai bộ đếm/ bộ định thời 8 bit với chế độ so sánh và chia tần số tách biệt. Một bộ định thời 16 bit với chế độ so sánh
- Bộ đếm thời gian thực (RTC) với bộ dao động tách biệt.
- Bộ điều chế độ rộng xung PWM 8 bit.
- Bộ biến đổi ADC bên trong 8 kênh 10 bit.
- 2 bộ USART nối tiếp lập trình được.
- Bộ định thời Watchdog lập trình được với bộ dao động trên chip.



❖ Sơ đồ chân:

- VCC: Điện áp nguồn nuôi. GND: Đất.
- Port B (PB0...PB7), Port C (PC0...PC7), Port D (PD0...PD7) là các port I/O 8 bit
- Port C cũng đóng vai trò như 8 đường địa chỉ cao từ A8 đến A15 khi kết nối bộ nhớ SRAM bên ngoài.
- Reset: Ngõ vào được đặt lại.
- AVCC: Cung cấp nguồn cho Port C và bộ chuyển đổi ADC hoạt động.
- AREF: Đây là chân điều chỉnh điện áp tham chiếu cho chuyển đổi A/D.
- XTAL1: Ngõ vào bộ khuếch đại đảo và ngõ vào mạch tạo xung nhịp bên ngoài.
- XTAL2: Ngõ ra bộ khuếch đại đảo.



Vi điều khiển Atmega8

- ❖ Arduino UNO có thể sử dụng 3 vi điều khiển họ 8bit AVR là ATmega8, ATmega168, ATmega328.

